



A novel unsupervised method for anomaly detection in time series based on statistical features for industrial predictive maintenance

Jesimar da Silva Arantes¹ · Márcio da Silva Arantes¹ · Herberth Birck Fröhlich² · Laure Siret³ · Renan Bonnard¹

Received: 26 April 2021 / Accepted: 19 August 2021 / Published online: 12 September 2021
© The Author(s), under exclusive licence to Springer Nature Switzerland AG 2021

Abstract

Industrial production processes are increasingly collecting data from machines in operation due to the cost reduction and popularization of sensor technologies. Valuable information is generated by using the right sensors and proper techniques on the machine's current state in operation. This extraction allows detecting whether the machine is operating in a degraded state and then, if necessary, interrupts its operation before it goes into a broken state. The detection of the abnormal behavior of a machine is relevant since detection at the right time can reduce financial costs due to machine breakdown and production downtime. This work proposes a novel unsupervised method to detect anomalies in industrial machines and interrupt their operation before this machine goes into a state of breakdown. The proposed method receives as input a set of time series data from several sensors. Using a collection of statistical features, it calculates an indicator that characterizes the failure's severity, issuing an alert to the machine operator if necessary. The method was evaluated in two benchmarks with known univariate data and two proprietary datasets with multivariate data. Conducted experiments revealed the low computational time spent on training and on evaluation. Overall results measured in Area Under the Receiver Operating Characteristic Curve (AUC) in Yahoo's benchmark were 89.3%; in Numenta, it was 70.3%, and in the two multivariate datasets evaluated, it was 92.4% and 91.2%. These high AUC values reveal the potential of the proposed method applied in predictive maintenance in a large soybean oil production industry.

Keywords Anomaly detection · Outlier detection · Time series · Feature extraction · Predictive maintenance · Industry 4.0

1 Introduction

Industry 4.0 revolution occurred mainly due to the increase in computational power, cheapening of sensing technologies, the evolution of Artificial Intelligence (AI) methods, and the fusion between the physical and digital world [15]. The 4.0 industry brings a series of advantages, such as increased productivity, cheaper production, increased interoperability, lower maintenance costs, and easier interaction between man and machine [20]. This work deals with the aspects related to reducing maintenance costs by developing a novel unsupervised method of anomaly detection in time series, aiming to assist predictive maintenance.

Many authors in the literature do not make a clear distinction between anomalies and outliers, as highlighted by [6]. An outlier is considered an abrupt change at a point that differs from its neighbors in the context of this work, possibly coming from some error in sensing or other error in the data entry. In turn, an anomaly is a change in a set of consecutive

✉ Jesimar da Silva Arantes
jesimar.arantes@sc.senai.br

Márcio da Silva Arantes
marcio.arantes@sc.senai.br

Herberth Birck Fröhlich
herberth.frohlich@sc.senai.br

Laure Siret
laure.siret@polymtl.ca

Renan Bonnard
renan.bonnard@sc.senai.br

¹ SENAI Institute of Embedded Systems, Av. Luiz Boiteux Piazza, Florianópolis, Santa Catarina, Brazil

² SENAI Institute of Embedded Systems, Federal University of Santa Catarina, Campus Universitário, Av. Luiz Boiteux Piazza, Florianópolis, Brazil

³ École Polytechnique Montréal, Montreal, QC, Canada

points that can lead a machine to a state of failure or breakage. Our novel method is capable of detecting anomalies and outliers. The method also makes it possible to distinguish local and global outliers. An outlier is local when it evades the pattern established by its neighbors. An outlier is global when it avoids the pattern concerning the entire data series [6].

As mentioned above, the occurrence of an anomaly is the expression of a deviation in the expected behavior of a machine. In turn, anomaly detection consists of the use of techniques that find these deviations in data series. There are two main ways of classifying anomaly detection methods: detection in temporal data and spatial data. The present work focuses on the detection of anomalies in temporal data (sequential). An outlier is a point that differs drastically from all the others, i.e., far away from the behavior considered standard. These points that differ from the others are often associated with sensing errors or errors in data entry. Our novel method can be applied to both anomaly detection and outlier detection, just by triggering the statistical features specific to each context.

Regarding predictive maintenance, the literature presents this subject with two main parts: diagnosis and prognosis. A diagnosis first detects, determines the causes, and then evaluates the machine's health status or system to prevent failures. A prognostic makes predictions about the machine's future state or system and predicts the Remaining Useful Life (RUL) [13]. The method proposed in this work focuses only on the fault diagnosis stage.

A maintenance procedure is based on four criteria: safety, economic cost, ease, and accuracy [5]. To ensure an optimal meet of these criteria, several types of maintenance strategies exist. Maintenance strategies are often classified into corrective, preventive (systematic and conditional), and predictive, as reported in [14,27]. The Corrective Maintenance (CrM) strategy is the simplest, as it only occurs when the machine has stopped working. CrM policies are economically advantageous for easy and short interventions. However, CrM policies are costly when applied to expensive systems or complex interventions due to the costs generated by downtime and new components. Systematic strategies are time-based only maintenance strategies. Conditional methods are only based on the machine condition, but the conditions' control and measurement can be done periodically. Preventive Maintenance (PvM) is a better strategy than CrM only if applied with the appropriate frequency. The main drawback of PvM is the unnecessary corrective actions performed due to a misunderstanding of the system, leading to an increasing in operational costs. Predictive Maintenance (PdM) is defined as an extension of PvM [13]. PdM uses predictive tools to estimate when maintenance actions will be necessary. Strategies of this type of maintenance involve continuous machine monitoring, allowing maintenance to be performed

only when necessary. If well applied, the PdM strategy brings financial savings since it reduces the number of breakdowns and improves survival duration to mechanical components [25]. The main drawback of PdM is the need for accurate data and good maturity of the system of information of the company [22].

The proposed method was called Anomaly and Outlier Detection (AOD), which is based on a set of statistical features that define acceptable limits for several variables that support univariate (coming from only one sensor) or multivariate (coming from more than one sensor) data. AOD aims to indicate anomalous behaviors that would possibly lead the machine to a maintenance need. That is an intrinsic need of industries since, as reported by [26], predictive maintenance can increase return on investment by up to 10 times, reduce maintenance costs from 25 to 30%, eliminate breakdowns from 70 to 75%, reduce downtime from 35 to 45%, and increase production from 20 to 25%.

This first step of diagnosis performed by AOD provides a unique way to reference different machine failures. Our method can replace or automatically complete fault trees or other failure identification techniques that are a prerequisite for any prognostic method. In addition, AOD also allows the identification of critical equipment states. A critical state is a state in which the equipment is not in a failure mode but in a state which a quality defect is observable in production. Identifying these states would improve the performance of maintenance intervention prediction during the prognostic step and thus improve the management of production quality.

The AOD method has been evaluated in two benchmarks with univariate data well known in the literature. The first benchmark evaluated is the Yahoo Outlier Benchmark (YOB), specific for detecting outliers in univariate time series [17]. The second dataset is the Numenta Anomaly Benchmark (NAB), typical for detecting anomalies in univariate time series [1,19]. Both datasets contain both artificial and real anomaly data. Actual anomalies are from various application fields, including Yahoo member login data, data stream, Amazon Web Services (AWS) server metrics, tweets volume, advertising click metrics, traffic data, and data from machines in operation. The evaluation methods took place in a proprietary dataset with synthetically generated multivariate data and on an actual case study with multivariate data from an industry in the vegetal oil production sector.

This article is organized as follows: Sect. 2 presents a brief literature review. Section 3 presents a description of the problem. Section 4 details the proposed method. Section 5 presents the computational results obtained. Finally, Sect. 6 closes this article with the final considerations.

2 Related works

There are several approaches in the literature to detect anomalies and outliers in time series. In the context of this work, we are interested in unsupervised techniques, as data labels are not always available for training in real applications. Hence, only Machine Learning (ML) and statistical methods will be considered in this review.

The authors in [7] present a literature review on ML techniques applied in predictive maintenance (PdM). This work shows that the main methods used in PdM, among others, are deep learning, greedy methods, and Bayesian methods. This work also states that these techniques can be applied to many types of equipment, such as turbines, engines, compressors, pumps, and fans. Another aspect raised by the work concerns the data used by the methods, where there is a preference for vibration data, acoustic signal, equipment temperature, electrical signal, and maintenance history.

The work of [21] presents the DeepAnT method that uses a predictor module of the series that uses Convolution Neural Network (CNN) and a module to detect anomalies, responsible for labeling the data as normal or abnormal. This unsupervised method learns the distribution of data and uses this information to predict the future point's behavior in a time series, and can be used on univariate or multivariate time series.

Still in the strategy of predicting future points to verify if the query point belongs to the expected distribution, the authors in [18] proposed the method EGADS, acronym for Extensible Generic Anomaly Detection System. EGADS performs the detection of outliers and detects points of change in univariate time series, and allows new methods to be added in each of these modules due to the way it was designed. This method has three main parts: a time series modeling module, an anomaly detection module, and an alert issuing module. This work has been validated over Yahoo data, with login data from members of that company.

In [12], the authors present the method S-H-ESD, an acronym for Seasonal Hybrid Extreme Studentized Deviate, which performs the detection of anomalies in univariate time series. S-H-ESD is a statistical method based on hypothesis testing and decomposition of time series into seasonality, trend, and noise. This work has been validated on Twitter data such as tweets per second and CPU utilization.

This work differs from the approaches of [12,18,21] because it makes the detection of anomalies and outliers based on the current analysis window and not on each data point. Still, the works of [12,18] are specific approaches to univariate time series.

More works can be cited in this context. In the work of [3], there is a multivariate control chart to detect potential environmental and ecological impacts. The authors in [8] proposed a method for detecting anomalies in time series

that uses wavelet decomposition with the Haar transform on the data. The work of [31] developed a technique based on AutoRegressive Integrated Moving Average (ARIMA) to detect potential attacks on computer networks. The work of [32] proposed a strategy for detection of anomalies in univariate time series based on self-adaptive statistical process control. The authors in [29] developed a method based on Hierarchical Temporal Memory (HTM) for anomaly detection in time series. The authors in [10] developed a technique based on the Long Short-Term Memory network (LSTM) and Gaussian Mixture Model (GMM).

The authors in [30] developed a spectral regression technique for the extraction of failure characteristics in rotating machines. These fault characteristics indicate the machine's current state in operation and can be used to detect anomalies. This work has used accelerometer sensors to capture bearing vibration data from rotating machines since the bearing behavior analysis is the most critical factor in detecting a failure in this machine type. The authors also extracted the accelerometer signal features in the time domain, frequency domain, and frequency-time domain. Table 1 summarizes the main technical characteristics of the papers reviewed in this study. Hence, the first compared technical characteristic is the object of the detection: anomalies, outliers or both of them, and then, the number of time series used (univariate or multivariate) and the type of learning used (supervised or unsupervised), and finally, the type of calculating strategies used by the authors.

On an other hand, several works on the literature have used several statistical features in the development of failure and anomaly detection techniques. The work of [30] has used the statistical features Root Mean Square (RMS), Square Root of Amplitude (SRA), Peak-Peak Value (PPV), Skewness Value (SV), Kurtosis Value (KV), Crest Factor (CF), Impulse Factor (IF), Margin Factor (MF), Shape Factor (SF), and Kurtosis Factor (KF) in the construction of a regression model for fault classification on accelerometer data. The authors in [24] used the features arithmetic mean (AMean), RMS, SRA, Standard Deviation (STD), Variance (VAR), PPV, Maximum Value (MaxV), Minimum Value (MinV), SV, KV, CF, IF, MF, SF, and KF in developing a method for detecting failures in bearings. In turn, the work of [11] used the features AMean, RMS, SRA, Mean Absolute Value (MVA), VAR, MaxV, SV, KV, CF, and SF in the extraction of characteristics of an induction motor over different power conditions for detection of broken bars. In the work of [9], the features AMean, STD, VAR, SV, and KV were used in the feature extraction of an induction motor for broken bar detection. The work of [28] used RMS, PPV, KV, and CF to build a diagnostic detection method applied in a gearbox. The work of [4] also highlights the statistical features RMS, PPV, CF, and KV give good results when used in engine failure analysis.

Table 1 Summary of the technical characteristics used in reviewed papers

Paper	Anomaly?	Outlier?	Nature of the method	Type of learning	Type of strategy
[21]	Yes	Yes	Multivariate	Unsupervised	CNN
[18]	No	Yes	Univariate	Unsupervised	Seasonal-trend decomposition
[12]	Yes	Yes	Univariate	Supervised	Seasonal-trend decomposition
[3]	Yes	No	Multivariate	Unsupervised	Control chart
[8]	Yes	No	Univariate	Unsupervised	Wavelet decomposition
[31]	Yes	No	Univariate	Unsupervised	ARIMA
[32]	Yes	No	Univariate	Unsupervised	Statistical technique
[29]	Yes	No	Multivariate	Supervised	HTM
[10]	Yes	No	Multivariate	Supervised	LSTM and GMM
[30]	Yes	No	Univariate	Supervised	Spectral regression
Our proposal	Yes	Yes	Multivariate	Unsupervised	Statistical technique

Table 2 Summary of different statistical features used by reviewed papers

Paper	Statistical features used Measures of central tendency	Measures of dispersion	Measures of coefficients in general
[30]	RMS and SRA	PPV	SV, KV, CF, IF, MF, SF and KF
[24]	AMean, RMS and SRA	STD, VAR and PPV	SV, KV, CF, IF, MF, SF, KF, MinV and MaxV
[11]	AMean, RMS, MVA and SRA	VAR	SV, KV, CF, SF and MaxV
[9]	AMean	STD and VAR	SV and KV
[28]	RMS	PPV	KV and CF
[4]	RMS	PPV	KV and CF
[12]	Median	MAD	—
Our proposal	AMean, RMS, MVA, SRA, MidR, Median, MidH, TriMean, IQM and TMean	STD, VAR, PPV, CV, MAV, IQR, IDR and QCD	SV, KV, CF, IF, MF, SF, KF, MinV, MaxV, SVBQ, KVBO, CVQ, Quartile1, Quartile3, Decile1 and Decile9

The works [4,9,11,24,28,30] have in common the fact that they use only nonrobust statistical estimators. This type of estimator is sensitive to anomalous data and outliers. Thus, this work also aims to evaluate the use of robust estimators in anomaly detection techniques. Statistical features, such as median, interquartile mean, Median Absolute Deviation (MAD), interquartile range, Skewness Value-Based Quartile (SVBQ), Kurtosis Value-Based Octile (KVBO), have also been implemented and evaluated. The robust estimators for asymmetry (SVBQ) and kurtosis (KVBO) were implemented based on the work of [16]. The idea of using robust estimators as median and MAD instead of non-rugged mean and standard deviation estimators was also explored in the work of [12], but with a very different implementation. Table 2 organizes different statistical features used by mentioned set of papers. The statistical features are categorized in three groups which are the measurement of central tendency, the measurements of dispersion and others. Some of the acronyms used in this table are detailed in Sect. 4.

It was not found on the literature unsupervised methods for time series that work on univariate and multivariate data to

detect anomalies and outliers using sliding windows, which runs near real time. In industrial applications, we are interested in unsupervised methods because we don't have the labels of the industrial data in most cases. It is also essential that the method runs in real time to ensure that the anomaly is detected in the current data window. Also, it is not necessary to know which points are anomalous but which windows are. The method developed can also be easily replicated for different industrial applications, where we have time series data that monitor the operating status of the machines.

3 Problem description

Consider an industrial production system composed of several machines that may or may not be the same. We will call the first machine to be monitored m_1 , the second machine to be monitored m_2 , and so on until the last machine m_Q . The machines must have sensors installed for artificial intelligence techniques to work. Therefore, consider that S_1^1 represents the first sensor installed on the machine m_1 , that

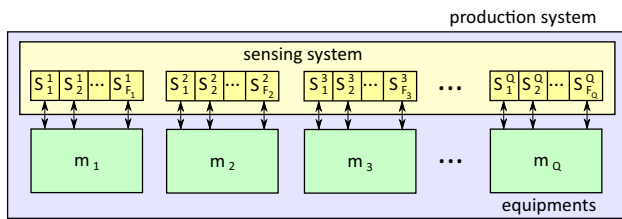


Fig. 1 Representation of an industrial production system composed of several machines and their sensors

S_2^1 represents the second sensor installed on the machine m_1 and so on until $S_{F_1}^1$ represents the last sensor installed on the machine m_1 . Similarly, the sensors S_1^2, S_2^2 up to $S_{F_2}^2$ represent the sensors installed on the machine m_2 . Finally, sensors S_1^Q, S_2^Q up to $S_{F_Q}^Q$ represent sensors installed in the machine m_Q . Figure 1 schematizes a part of the industrial production system where the plant machines and the sensors installed on each of these machines are highlighted.

The output of interest of a sensed machine is time-spaced data, characterizing a time series. Thus, this series, s , is defined as a sequence of measurements of a variable at different points in time and described mathematically in Eq. (1).

$$s = [(x_1, t_1), (x_2, t_2), (x_3, t_3), \dots, (x_N, t_N)] \quad (1)$$

This series is composed of a collection of ordered pairs (x_r, t_r) , where x_r represents the value of the series at the instant of time t_r . In this series, the value N represents the total amount of data, so $r \in \{1, 2, 3, \dots, N\}$. One of the characteristics of time series is that the value t_r is strictly incremental, that is, if $r_1 < r_2$ implies that $t_{r_1} < t_{r_2}$. The vast majority of time series models consider $\Delta t = t_r - t_{r-1}$ constant, i.e., the time interval between any two samples is always the same. The method proposed in this work also considers Δt constant, and therefore, this same time series can be written as $s = [x_1, x_2, x_3, \dots, x_N]$. This work will manipulate several time series, each coming from a different sensor, so it is necessary to distinguish the data series of a i sensor, as shown in Eq. (2).

$$s_i = [x_{i,1}, x_{i,2}, x_{i,3}, \dots, x_{i,n}] \quad \forall i \in \{1, 2, 3, \dots, F\} \quad (2)$$

Still, about Eq. (2), there is a total of F sensors in a given machine. Most of the times, different sensors have different acquisition frequencies, that is, different values of Δt . Data from all sensors were preprocessed to ensure that Δt would be the same for all of them. Such a procedure usually leads to a reduction in the dataset size but brings a more coherent analysis. However, although performed, the synchronization methods were not implemented or were not needed in specific benchmarks.

Although Fig. 1 presents several machines that can use the proposed method for anomaly detection and predictive maintenance assistance, the problem can be drastically simplified by applying one instance of the technique to each of the machines independently. Using one instance of the technique like this can be done because most industrial machines' defects occur independently of each other. For example, if two machines have independent functions and no common support mechanism, then the probability of failure is also independent [5]. This way of modeling simplifies both the model and the problem description. For example, Eq. (2) will not need an index to identify which machine belongs to the time series.

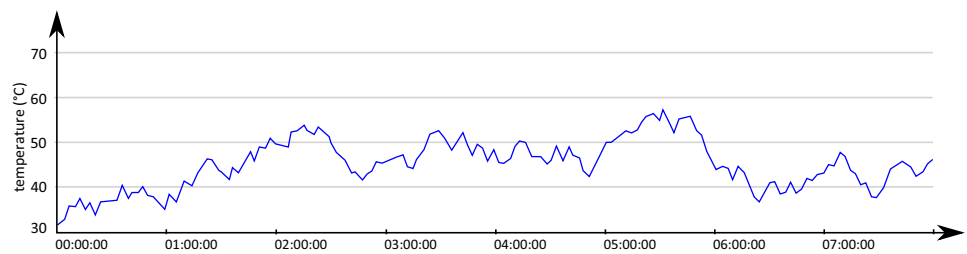
Let's consider the example of Fig. 2 to illustrate an anomaly in a time series that would lead to a machine breakdown. Figure 2a shows a hypothetical time series representing the engine temperature of any machine during 8 h of operation. This series presents some fluctuations where the temperature oscillates between 30 °C and 60 °C. These fluctuations are considered normal for the machine in question. Figure 2b shows the same example, but from 5 h, the engine temperature starts to rise until around 6 h and 12 min the machine overheats to the point that it even breaks down (possibly the engine stops running). After this event, the machine interrupts its operation, and the temperature decreases approximately linearly.

The main idea of the AOD method is to use the time series data and detect the anomaly in the data. What often occurs is that machines in operation do not suddenly break. Moments before the break, the sensors' data already indicate that something is out of the pattern. For example, look in Fig. 2b at the window demarcated by dashed lines. An algorithm would notice that the machine was having abnormal behavior due to its high temperature before the breaking point.

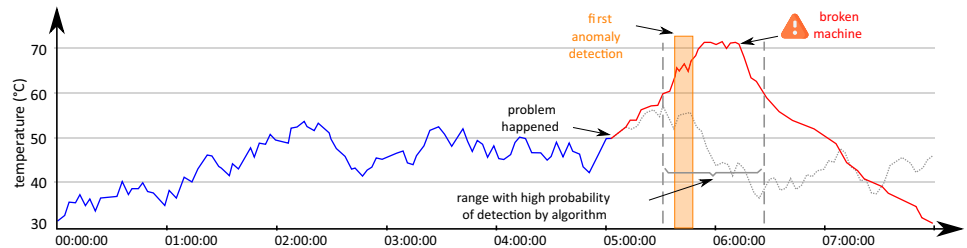
Figure 2c shows the outlier detection procedure on hypothetical data. In this example, there were two outlier detections in different windows. As the outlier value generally reflects an incorrect sensor reading or even some data acquisition failure, the outlier detection does not lead to a breakage of the machine in question. Due to its sudden characteristic, the outlier is difficult to predict, but it is relatively easy to detect, as it often causes great disturbance to local data. Because of this characteristic, the outlier has little impact on the area of predictive maintenance.

Figure 2d shows a time series with a behavior change point where a new concept (scenario) of operation is established. The change in signal occurs due to a new fan installed in the monitored machine that reduces the machine's average temperature considerably by about 18 °C. Thus, the machine that operated with temperatures ranging from 30 to 60 °C now has temperatures in the range of 12 °C and 42 °C. Concept changes like this require artificial intelligence techniques to be relearned continuously, as reported by [18]. If training was

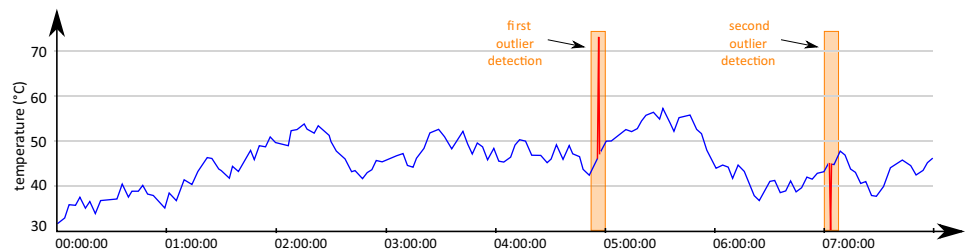
Fig. 2 Examples of time series that represent the temperature behavior of a hypothetical machine without anomaly and with anomaly



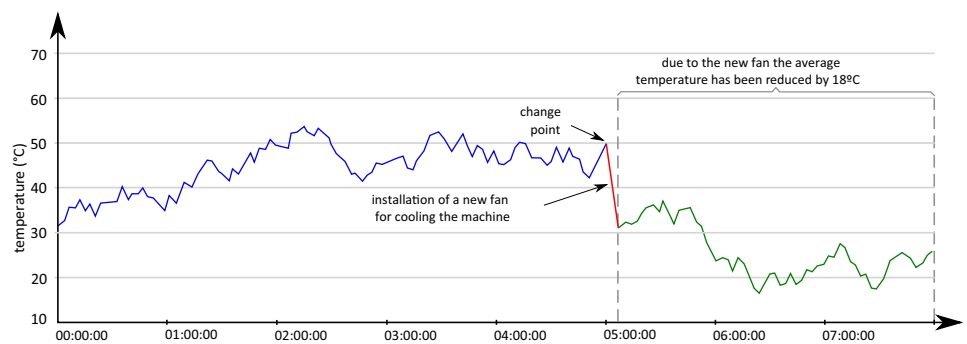
(a) Example of univariate time series without anomaly.



(b) Example of univariate time series with anomaly. The detection of the anomaly is estimated.



(c) Example of univariate time series with outliers. The detection of outliers is estimated.



(d) Example of univariate time series without anomaly with point of change.

established in one concept and evaluated another concept, then the number of false positives and false negatives could increase considerably.

4 Methods

This method uses a set of statistical features to synthesize the variation limits of an input signal. There are other possibilities for feature extraction based on deep learning, such as one-dimensional convolutional neural networks. This

approach for addressing data features was considered at first, but the large volume of data and the frequent need to update the models would make in-plant processing unfeasible. Therefore, this extraction by statistical variables directly proved to be an active and reasonable alternative to deep learning. These statistical features can be separated into groups, representing central tendency measures, dispersion, coefficients in general, and its derivatives. These groups are the same as mentioned in Table 2. Some of these features have the same unit of measurement as the input data, others have no measurement units, and others have a different unit than the input. Consider the following terminology:

- $\mathbf{x}$: represents the data vector;
- x_i : represents the i th element of the data vector $\mathbf{x}$;
- n : represents the number of elements of $\mathbf{x}$;
- $\bar{\mathbf{x}}$: represents the arithmetic mean of the vector $\mathbf{x}$;
- σ : represents the standard deviation of the data $\mathbf{x}$;
- $\hat{\mathbf{x}}$: vector represents the ordered data vector $\mathbf{x}$;
- $\hat{x}_i$: represents the i th element of $\hat{\mathbf{x}}$;
- p : represents a probability value between $0 \leq p \leq 1$;
- $\mathbf{x}'$: represents the vector of data in the domain of the derivative.

Also consider the following functions used in the equations:

- $\min(\mathbf{x})$: calculates the smallest value of $\mathbf{x}$;
- $\max(\mathbf{x})$: calculates the largest value of $\mathbf{x}$;
- $\text{median}(\mathbf{x})$: calculates the median value of $\mathbf{x}$;
- $\text{decile}(k, \mathbf{x})$: calculates the k th decile of $\mathbf{x}$;
- $\text{qtl}(k, \mathbf{x})$: calculates the k th quartile of $\mathbf{x}$;
- $\text{otl}(k, \mathbf{x})$: calculates the k th octile of $\mathbf{x}$.

4.1 Statistical features

A central tendency measure is a value that represents the idea of central value in a data set. The method developed supports a total of 10 central tendency features that may be selected or not: Arithmetic Mean (AMean), Root Mean Square (RMS), Mean Value Absolute (MVA), Square Root of Amplitude (SRA), MidRange (MidR), Median, MidHinge (MidH), TriMean, InterQuartile Mean (IQM), and Truncated Mean (TMean).

A measure of dispersion is a value that represents the idea of how elongated or narrow the distribution of a data set is. A total of 8 features of dispersion may be selected: Standard Deviation (STD), Variance (VAR), Peak–Peak Value (PPV), Coefficient of Variation (CV), Median Absolute Deviation (MAD), InterQuartile Range (IQR), InterDecile Range (IDR), and Quartile Coefficient of Dispersion (QCD).

In general, a coefficient measure is a measure of form, an index, or even a measure of extremes of a data set. The method developed supports a total of 16 features of coefficients that may be selected, being them: Skewness Value (SV), Kurtosis Value (KV), Crest Factor (CF), Impulse Factor (IF), Margin Factor (MF), Shape Factor (SF), Kurtosis Factor (KF), Minimum Value (MinV), Maximum Value (MaxV), Skewness Value-Based Quartile (SVBQ), Kurtosis Value-Based Octile (KVBO), Coefficient of Variation in the Quartile (CVQ), first quartile (quartile1), third quartile (quartile3), first decile (decile1), and ninth decile (decile9).

Finally, a measure in the derivative domain is one of the above features calculated on the series after applying discrete derivative. The method developed supports a total of 34 features in the derivative domain. Some of the supported measures are Derivative of Arithmetic Mean (DAMean), Derivative of STD (DSTD), Derivative of SV (DSV), Derivative of KV (DKV), Derivative of Median (DMedian), Derivative of IQR (DIQR), Derivative of SVBQ (DSVBQ), and Derivative of KVBO (DKVBO).

4.2 Extracted features summary

Table 3 summarizes the previously mentioned features and shows their computational time complexity. It can be seen that there are some features with linear complexity $\mathcal{O}(n)$, where n is the total number of input observations. For example, the arithmetic mean has linear complexity in n . This way, one can calculate this metric with constant value times n of runs over the input data. There are also some metrics that have log-linear complexity or $\mathcal{O}(n \log(n))$. For example, the median is a metric with log-linear complexity, and this is explained by the fact that it is necessary to sort the data to then return to the median.

Table 3 also lists the features of their robustness against outliers, column “Robust.” A feature is robust when it is not or is barely affected by outliers. Another characteristic of robust features is that their performance is suitable for a wide range of probability distributions beyond the normal distribution [23]. Being robust for several distributions is essential since most time-series distributions in real applications do not follow the normal distribution, as stated in [12].

The choice of which features are of interest depends exclusively on the application. If we want this method to be applied to outlier detection problems, nonrobust estimators should be selected to highlight these points more effectively. On the other hand, if the interest is to ignore the outliers and detect only the anomalies, robust estimators should be selected. If the time series is stationary, then the central tendency metrics should be chosen to show possible anomalies when the series escapes stationarity’s established pattern. Similarly, if the time series is non-stationary, then the central tendency

Table 3 Statistical features supported by AOD with its respective complexity and robustness to the existence of outliers

Domain	Feature name	Complexity	Robust	Equation
Measures of central tendency	Arithmetic mean	$\mathcal{O}(n)$		$M_{AMean} = \frac{1}{n} \cdot \sum_{i=1}^n x_i$
	Root mean square	$\mathcal{O}(n)$		$M_{RMS} = \left(\frac{1}{n} \cdot \sum_{i=1}^n x_i^2 \right)^{\frac{1}{2}}$
	Mean value absolute	$\mathcal{O}(n)$		$M_{MVA} = \frac{1}{n} \cdot \sum_{i=1}^n x_i $
	Square root of amplitude	$\mathcal{O}(n)$		$M_{SRA} = \left(\frac{1}{n} \cdot \sum_{i=1}^n \sqrt{ x_i } \right)^2$
	MidRange	$\mathcal{O}(n)$		$M_{MidR} = \frac{\min(x) + \max(x)}{2}$
	Median	$\mathcal{O}(n \log(n))$		$M_{Median} = \text{median}(x)$
	MidHinge	$\mathcal{O}(n \log(n))$		$M_{MidH} = \frac{\text{qtl}(1, x) + \text{qtl}(3, x)}{2}$
	TriMean	$\mathcal{O}(n \log(n))$		$M_{TriMean} = \frac{M_{Median}(x) + M_{MidH}(x)}{2}$
	InterQuartile mean	$\mathcal{O}(n \log(n))$		$M_{IQM} = \frac{2}{n} \cdot \sum_{i=0}^{0.75n} \hat{x}_i$
Measures of dispersion	Truncated mean	$\mathcal{O}(n \log(n))$		$M_{TMean} = \frac{1}{(1-2p)n} \cdot \sum_{i=p \cdot n+1}^{(1-p)n} \hat{x}_i$
	Standard deviation	$\mathcal{O}(n)$		$M_{STD} = \left(\frac{1}{n} \cdot \sum_{i=1}^n (x_i - \bar{x})^2 \right)^{\frac{1}{2}}$
	Variance	$\mathcal{O}(n)$		$M_{VAR} = \frac{1}{n} \cdot \sum_{i=1}^n (x_i - \bar{x})^2$
	Peak-peak value	$\mathcal{O}(n)$		$M_{ppv} = \max(x) - \min(x)$
	Coefficient of variation	$\mathcal{O}(n)$		$M_{CV} = M_{STD}(x) / M_{AMean}(x)$
	Median absolute deviation	$\mathcal{O}(n \log(n))$		$M_{MAD} = 1.4826 \cdot \text{median}(x_i - \text{median}(x))$ $i = 1, \dots, n$
	InterQuartile range	$\mathcal{O}(n \log(n))$		$M_{IQR} = \text{qtl}(3, x) - \text{qtl}(1, x)$
	InterDecile range	$\mathcal{O}(n \log(n))$		$M_{IDR} = \text{decile}(9, x) - \text{decile}(1, x)$
	Quartile coefficient of dispersion	$\mathcal{O}(n \log(n))$		$M_{QCD} = \frac{\text{qtl}(3, x) - \text{qtl}(1, x)}{\text{qtl}(3, x) + \text{qtl}(1, x)}$
Measures of coefficients in general	Skewness value	$\mathcal{O}(n)$		$M_{SV} = \frac{1}{n} \cdot \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^3$
	Kurtosis value	$\mathcal{O}(n)$		$M_{KV} = \frac{1}{n} \cdot \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^4 - 3$
	Crest factor	$\mathcal{O}(n)$		$M_{CF} = \max(x) / M_{RMS}(x)$
	Impulse factor	$\mathcal{O}(n)$		$M_{IF} = \max(x) / M_{MVA}(x)$
	Margin factor	$\mathcal{O}(n)$		$M_{MF} = \max(x) / M_{SRA}(x)$
	Shape factor	$\mathcal{O}(n)$		$M_{SF} = M_{RMS}(x) / M_{MVA}(x)$
	Kurtosis factor	$\mathcal{O}(n)$		$M_{KF} = M_{KV}(x) / \left(\frac{1}{n} \cdot \sum_{i=1}^n x_i^2 \right)^2$
	Minimum value	$\mathcal{O}(n)$		$M_{MinV} = \min(x)$
	Maximum value	$\mathcal{O}(n)$		$M_{MaxV} = \max(x)$
	Skewness value-based quartile	$\mathcal{O}(n \log(n))$		$M_{SVBQ} = \frac{\text{qtl}(3, x) + \text{qtl}(1, x) - 2 \cdot \text{qtl}(2, x)}{\text{qtl}(3, x) - \text{qtl}(1, x)}$
	Kurtosis value-based octile	$\mathcal{O}(n \log(n))$		$M_{KVBO} = \frac{\text{oct}(7, x) - \text{oct}(5, x) + \text{oct}(3, x) - \text{oct}(1, x)}{\text{oct}(6, x) - \text{oct}(2, x)} - 1.23$
	Coefficient of variation in the quartile	$\mathcal{O}(n \log(n))$		$M_{CVQ} = M_{IQR}(x) / M_{IQM}(x)$
	Quartile1	$\mathcal{O}(n \log(n))$		$M_{Quartile1} = \text{qtl}(1, x)$
	Quartile3	$\mathcal{O}(n \log(n))$		$M_{Quartile3} = \text{qtl}(3, x)$
	Decile1	$\mathcal{O}(n \log(n))$		$M_{Decile1} = \text{decile}(1, x)$
	Decile9	$\mathcal{O}(n \log(n))$		$M_{Decile9} = \text{decile}(9, x)$
Measures in the domain of the derivative	Derivative of AMean	$\mathcal{O}(n)$		$M_{DAMean} = M_{AMean}(x')$
	Derivative of STD	$\mathcal{O}(n)$		$M_{DSTD} = M_{STD}(x')$
	Derivative of SV	$\mathcal{O}(n)$		$M_{DSV} = M_{SV}(x')$
	Derivative of KV	$\mathcal{O}(n)$		$M_{DKV} = M_{KV}(x')$
	Derivative of median	$\mathcal{O}(n \log(n))$		$M_{DMedian} = M_{Median}(x')$
	Derivative of IQR	$\mathcal{O}(n \log(n))$		$M_{DIQR} = M_{IQR}(x')$
	Derivative of SVBQ	$\mathcal{O}(n \log(n))$		$M_{DSVBQ} = M_{SVBQ}(x')$
	Derivative of KVBO	$\mathcal{O}(n \log(n))$		$M_{DKVBO} = M_{KVBO}(x')$

metrics should not be used, as the series does not oscillate around a regular interval. In general, metrics taken in the derivative domain emphasize characteristics not highlighted

in a standard signal, such as the roughness of a signal measured through the dispersion measures. Finally, the derivative domain metrics maintain the signal's temporality aspect,

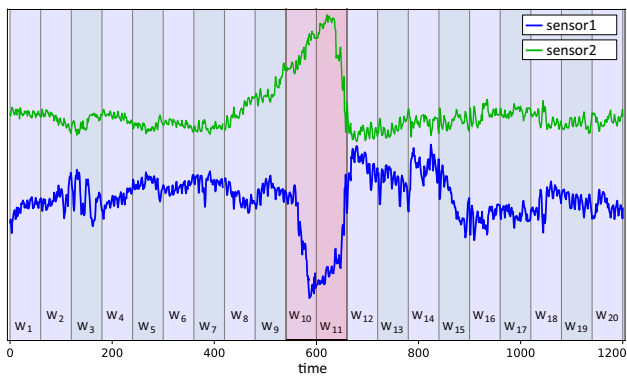


Fig. 3 Hypothetical data of two sensors connected to the same machine

which does not occur with the metrics calculated using the standard signal.

4.3 Learning stage algorithm

AOD, like many artificial intelligence techniques, has two stages, training and testing. Since the proposed method is unsupervised, it is not necessary to have the observation labels. The only requirement during training is that the data series come from a machine under normal operating conditions. If there are data that represent anomalies or outliers, these must be removed, causing the algorithm to learn only the machine's regular operation. A strategy to remove any anomalies from the training set has also been developed since it is impossible to guarantee that the training's input data are related to normal operating conditions. Regarding the number of observations, the higher the volume of data in the training set, the better. Nevertheless, the results were considered of good quality, even with little data in the training set. A sliding window is used to define the features extraction region and subsequent analysis during testing. The justification for using a data sliding window is because the window allows the calculation of a set of features that require multiple data. Also, the window reveals the signal characteristic better than the techniques that consider only one data point at a time. The detected outliers and anomalies are thus associated with one or more of these windows, composed of an arbitrary number of points in the time series.

Before we present the training algorithm, let's consider Fig. 3, which illustrates some hypothetical data for training related to two different sensor series but which acquire data from the same machine. In this figure, each of the time series was divided into 20 sub-windows with 60 points each, named W_1 , W_2 to W_{20} . Each of these windows carries certain information but most often shows the normal operating behavior of the machine. However, in the W_{10} and W_{11} windows, the machine has presented an anomaly, bringing the need to remove these windows from the training, as we will

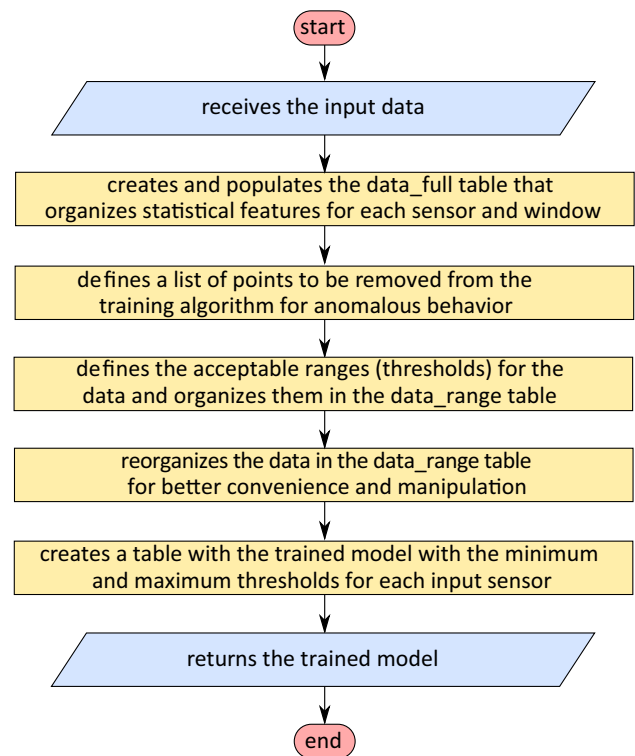


Fig. 4 Simplified flowchart of Algorithm 1

see later. During the training, in each sub-windows, the statistical features listed in Table 3 are calculated. The largest and smallest values obtained for each of the chosen features and each of the sensors are stored. These values represent the trained learning model.

A basic flowchart is presented in Fig. 4 with the main steps performed in the training algorithm. Figure 4 synthesizes the contents of Algorithm 1 which also includes Algorithm 2 as the second block of the flowchart (definition of the list of points to be removed). Algorithm 1 presents the main steps to acquire the features limits selected from Table 3. In this algorithm, *df_train* table contains the data collected by the sensors over a long period of time. The data from *df_train* are used in the training stage. *Data_full* table organizes the statistical features calculated for each sensor and each of the training windows. *Data_range* table contains the minimum and maximum values of all statistical features extracted from each sensor and each feature. Finally, *data_to_df* table stores the learned model and consists of a reorganization of the data table. Algorithm 1 used in the training stage is described below.

Algorithm 2 presents the method for removing windows with potential anomalies in training. This method returns a list of all window indexes that must be removed from the training set. The criteria used to remove windows are based on a measure of distance calculated between each window and the data centroid. The windows that move away from

Algorithm 1: Training stage pseudocode.

```

1 begin
   Input: list_sensors, list_metrics, df_train, window_size, num_window,
   cut_factor
   Output: df_model
2  $F \leftarrow | \text{list\_sensors} |$   $J \leftarrow | \text{list\_metrics} |$   $K \leftarrow \text{num\_window}$   $N \leftarrow$ 
   window_size  $\text{data\_full} \leftarrow \text{create\_array}(K, F, J)$  // stores data
   in the data_full table that contains the
   features
3 for  $k \leftarrow 1$  to  $K$  do
4   for  $i \leftarrow 1$  to  $F$  do
5      $\text{data\_sensor} \leftarrow \text{df\_train}[\text{list\_sensors}[i]].\text{values}$  // selects
     data by window
6      $\text{serie} \leftarrow \text{data\_sensor}[k \times N : (k+1) \times N]$  for  $j \leftarrow 1$  to  $J$  do
7        $\text{data\_full}[k][i][j] \leftarrow \text{calc\_metric\_stats}(\text{serie}, \text{list\_metrics}[j])$ 
8  $\text{list\_idx\_cut} \leftarrow \text{Algorithm2}(F, J, K, \text{cut\_factor}, \text{data\_full})$   $\text{data\_range} \leftarrow \emptyset$ 
   // defines the possible ranges of the data in
   the data_range table
9 for  $i \leftarrow 1$  to  $F$  do
   // list of metrics with minimum values
10   $\text{lmtr\_min} \leftarrow \emptyset$  // list of metrics with maximum
   values
11   $\text{lmtr\_max} \leftarrow \emptyset$  for  $j \leftarrow 1$  to  $J$  do
12     $\text{list\_metric\_per\_window} \leftarrow \emptyset$ 
13    for  $k \leftarrow 1$  to  $K$  do
14      if  $k \notin \text{list\_idx\_cut}$  then
15         $\text{list\_metric\_per\_window.add}(\text{data\_full}[k][i][j])$ 
16       $\text{lmtr\_min.add}(\min(\text{list\_metric\_per\_window}))$ 
17       $\text{lmtr\_max.add}(\max(\text{list\_metric\_per\_window}))$ 
   // stores a tuple with list data
18   $\text{data\_range.add}((\text{lmtr\_min}, \text{lmtr\_max}))$ 
19  $\text{data\_to\_df} \leftarrow \emptyset$ 
   // rearranges the data in the data_range table
   for better convenience
20 for  $j \leftarrow 1$  to  $J$  do
21   $\text{list\_min\_values\_per\_sensor} \leftarrow \emptyset$ 
22  for  $i \leftarrow 1$  to  $F$  do
23     $\text{list\_min\_values\_per\_sensor.add}(\text{data\_range}[i][0][j])$ 
24   $\text{data\_to\_df.add}(\text{list\_min\_values\_per\_sensor})$ 
25   $\text{list\_max\_values\_per\_sensor} \leftarrow \emptyset$ 
26  for  $i \leftarrow 1$  to  $F$  do
27     $\text{list\_max\_values\_per\_sensor.add}(\text{data\_range}[i][1][j])$ 
28   $\text{data\_to\_df.add}(\text{list\_max\_values\_per\_sensor})$ 
29  $\text{df\_model} \leftarrow \text{create\_dataframe}(\text{data\_to\_df}, \text{list\_sensors}, \text{list\_metrics})$ 
30 return  $\text{df\_model}$ 

```

the data centroid tend to be the ones in which the machine presents a problem, so they should be removed. For example, if you run the window removal algorithm on the data of Fig. 3 using a *cut_factor* equal to 10%, then two windows will be removed 20 windows in total. The windows removed would be W_{10} and W_{11} which are the two windows that most deviate from the established operation pattern. The list of points (*lpoints*) has a total of K points, where each point belongs to the real coordinate space $\mathbb{R}^D = \mathbb{R}^{F \cdot J}$. The same occurs with the list of the standardized points list. The centroid is a point in the real coordinate space $\mathbb{R}^D$ that stores the average behavior of the metrics of each of the data window sensors. Thus, the points that are furthest from the centroid are removed from the training set.

The time complexity of the algorithm's training phase is given by $\mathcal{O}(K \cdot F \cdot J \cdot N \cdot \log(N))$. Thus, we can see that the training is linear concerning the number of windows (K), the

number of sensors (F), and the number of metrics (J). The training is log-linear concerning the number of data points per window (N). This complexity is almost linear in relation to the input parameters, this makes the training phase extremely fast and appropriate for Big Data applications.

Algorithm 2: Step to remove windows with possible anomalies.

```

1 begin
   Input:  $F, J, K, \text{cut\_factor}, \text{data\_full}$ 
   Output:  $\text{list\_idx\_cut}$ 
2  $\text{list\_idx\_cut} \leftarrow \emptyset$ 
   // amount windows that will be removed from
   training
3  $\text{cut} \leftarrow \text{cut\_factor} \times K$ 
4 if  $\text{cut} > 0$  then
5    $D \leftarrow F \times J$ 
6    $\text{lpoints} \leftarrow \emptyset$ 
7   for  $k \leftarrow 1$  to  $K$  do
   // create an array with  $D$  dimension
8      $\text{point} \leftarrow \text{create\_array}(D)$ 
9      $d \leftarrow 1$ 
10    for  $i \leftarrow 1$  to  $F$  do
11      for  $j \leftarrow 1$  to  $J$  do
12         $\text{point}[d] \leftarrow \text{data\_full}[k][i][j]$ 
13         $d \leftarrow d + 1$ 
14     $\text{lpoints.add}(\text{point})$ 
   // create array initialized with -infinity
15   $\text{maxpoint} \leftarrow \text{create\_array}(D, -\text{infinity})$ 
   // create array initialized with +infinity
16   $\text{minpoint} \leftarrow \text{create\_array}(D, +\text{infinity})$ 
17  for  $k \leftarrow 1$  to  $K$  do
18    for  $d \leftarrow 1$  to  $D$  do
19       $\text{maxpoint}[d] \leftarrow \max(\text{lpoints}[k][d], \text{maxpoint}[d])$ 
20       $\text{minpoint}[d] \leftarrow \min(\text{lpoints}[k][d], \text{minpoint}[d])$ 
21   $\text{lpoints\_normalized} \leftarrow \emptyset$ 
22  for  $k \leftarrow 1$  to  $K$  do
23     $\text{point} \leftarrow \text{create\_array}(D)$ 
24    for  $d \leftarrow 1$  to  $D$  do
25       $\text{point}[d] \leftarrow (\text{lpoints}[k][d] - \text{minpoint}[d]) / (\text{maxpoint}[d] - \text{minpoint}[d])$ 
26     $\text{lpoints\_normalized.add}(\text{point})$ 
27   $\text{centroid} \leftarrow \text{create\_array}(D)$ 
28  for  $k \leftarrow 1$  to  $K$  do
29    for  $d \leftarrow 1$  to  $D$  do
30       $\text{centroid}[d] \leftarrow \text{centroid}[d] + (\text{lpoints\_normalized}[k][d]/K)$ 
31   $\text{list\_dist\_centroid} \leftarrow \emptyset$ 
32  for  $k \leftarrow 1$  to  $K$  do
33     $\text{list\_dist\_centroid.add}(\text{euclidian\_dist}(\text{lpoints\_normalized}[k], \text{centroid}))$ 
   // returns the indexes that sort array
34   $\text{order} \leftarrow \text{argsort}(\text{list\_dist\_centroid})$ 
35  for  $c \leftarrow 1$  to  $\text{cut}$  do
   // add indexes farthest centroid
36     $\text{list\_idx\_cut.add}(\text{order}[\text{length}(\text{order}) - c])$ 
37 return  $\text{list\_idx\_cut}$ 

```

4.4 Test stage algorithm

This phase occurs after the extraction of the parameters with the method's application in a new data set that may contain anomalies and outliers. Figure 5 shows the flowchart for the method of evaluating that looks for possible anomalies.

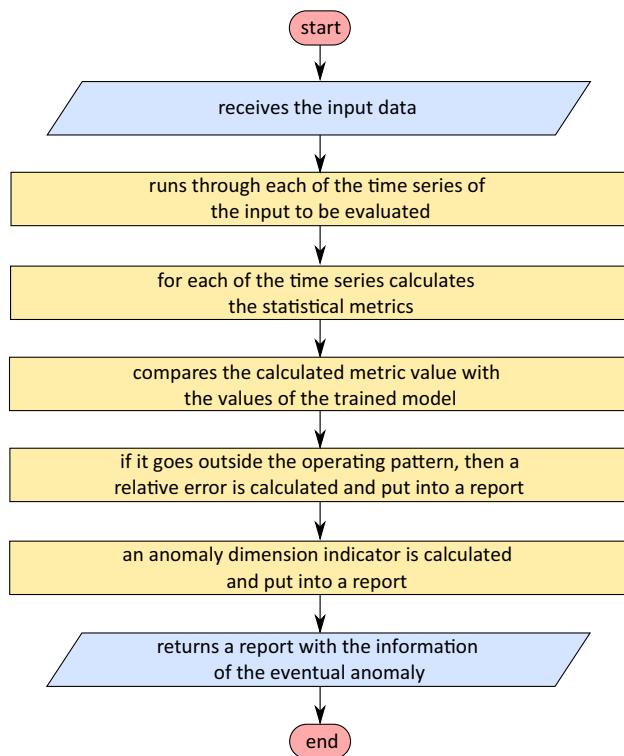


Fig. 5 Simplified flowchart of Algorithm 3

Algorithm 3 shows the idea behind the evaluation method to identify a possible anomaly or outlier in the input data. An important aspect related to the detection of anomalies is the reduction in noise in the data. For this purpose, the need arises not only to select robust features but also to work with sliding windows. The anomaly or outlier detection is then performed for the window in question and not at every point of the data that make up that window. When pointing, for example, that a particular window contains an outlier, the method does not highlight at which point of the window the outlier is but signals the window as a whole.

The time complexity of the evaluation phase of the algorithm is given by $\mathcal{O}(F \cdot J \cdot N \cdot \log(N))$. Thus, the evaluation's complexity is similar to the complexity of the training, except for the absence of the number of windows factor (K). This point is justified because the evaluation is done only for the current window. The fact that the complexities, both in training and evaluation, are almost linear implies that the method performs the training and the evaluation very quickly. The typical values of the parameters K , F , J and N of AOD are: $K[10, 10000]$, $F[1, 20]$, $J[4, 20]$, and $N[5, 500]$.

5 Results

The AOD was evaluated in two univariate public benchmarks and two proprietary multivariate datasets. The first evaluated dataset, Yahoo Outlier Benchmark (YOB), is used to

Algorithm 3: Test stage pseudocode.

```

1 begin
  Input: list_sensors, list_metrics, current_window, df_model
  Output: report
  F ← | list_sensors |
  J ← | list_metrics |
  report ← ∅
  sum_error_rel ← 0
  for name_sensor in list_sensors do
    list_sensors_affecteds ← ∅
    serie ← current_window[name_sensor].values
    for j ← 1 to J do
      metric ← calc_metric_stats(serie, list_metrics[j])
      v_min ← df_model[name_sensor, 'min' + list_metrics[j]]
      v_max ← df_model[name_sensor, 'max' + list_metrics[j]]
      if metric < v_min then
        error_rel_normalized ← | (metric - v_min) / (v_max - v_min) |
        sum_error_rel ← sum_error_rel + error_rel_normalized
        list_sensor_affecteds.add('min' + list_metrics[j])
        report.count_metrics ← report.count_metrics + 1
      if metric > v_max then
        error_rel_normalized ← | (metric - v_max) / (v_max - v_min) - 1 |
        sum_error_rel ← sum_error_rel + error_rel_normalized
        list_sensor_affecteds.add('max' + list_metrics[j])
        report.count_metrics ← report.count_metrics + 1
    if report.count_metrics ≥ 1 then
      report.count_sensors_affecteds ← report.count_sensors_affecteds + 1
      report.dict_failures[name_sensor] ← list_metrics_affecteds
  if report.count_metrics ≥ 1 then
    report.ind_dim ← sum_error_rel × 100 / (J × F)
  return report

```

detect outliers in univariate time series. The second evaluated benchmark, Numenta Anomaly Benchmark (NAB), is used for anomalies and outliers detection in univariate time series. The third is a dataset with multivariate data generated synthetically using the AGOTS tool. The fourth dataset consists of multivariate data from a case study of a vegetal oil production industry. Both YOB and NAB benchmarks have synthetic and real data on different application domains. This section's results are separated according to the benchmarks evaluated: YOB, NAB, synthetic dataset, and case study.

The metrics used to evaluate the results are: Recall (also known as sensitivity), Precision, Accuracy, F1-score, and Area Under the Receiver Operating Characteristic Curve (AUC). These metrics are commonly used to report the efficiency of algorithms in the context of anomaly and outlier detection [2, 18, 21, 27]. The equations and a short description of these metrics are presented in Table 4, where TP, FP, FN, and TN are acronyms for True Positive, False Positive, False Negative, and True Negative.

5.1 Evaluation on YOB: univariate

Here are described the computational results obtained on Yahoo Outlier Benchmark (YOB).¹ This data set contains

¹ <https://webscope.sandbox.yahoo.com/catalog.php?datatype>.

Table 4 Set of metrics used to evaluate the results

Metric	Equation	Description
Recall	$R_{\text{rec}} = \frac{\text{TP}}{\text{TP} + \text{FN}}$	Metric that highlights the percentage of hits in relation to the true universe
Precision	$R_{\text{pre}} = \frac{\text{TP}}{\text{TP} + \text{FP}}$	Metric that highlights the percentage of hits in relation to affirmative prediction of the truth
Accuracy	$R_{\text{acc}} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$	Metric that highlights the percentage of hits in relation to all the statements made
F1-score	$R_{\text{F1}} = 2 \cdot \frac{R_{\text{rec}} \cdot R_{\text{pre}}}{R_{\text{rec}} + R_{\text{pre}}}$	Metric that performs the average harmonic between the metrics Recall and Precision. Thus, a weighting is made between these two metrics
AUC	$R_{\text{auc}} = \int_0^1 \text{TPR}(\text{FPR}) \, d\text{FPR}$	Metric that calculates the area below the True Positive Rate (TPR) and False Positive Rate (FPR) curve

367 univariate time series with labeled outliers and is subdivided into four sub-benchmarks, named A1, A2, A3, and A4. The A1 sub-benchmark includes 67 time series with actual data. The A2, A3, and A4 sub-benchmarks contain 300 time series with synthetic data, totaling 100 time series per sub-benchmark. The A2 and A3 sub-benchmarks have only outliers, while the A4 sub-benchmark contains both outliers and change points.

Label information was only used to evaluate the method's performance due to AOD unsupervised nature. All equations used for such an evaluation are shown in Table 4. In evaluating this benchmark, we are interested only in outlier detection because this benchmark has no anomalies. Thus, the statistical indicators used were: M_{STD} , M_{PPV} , M_{DSTD} , M_{DPPV} . So we have that $J = 4$, and the list of metrics of the algorithms 1 and 3 becomes $\text{list_metrics} = [M_{\text{STD}}, M_{\text{PPV}}, M_{\text{DSTD}}, M_{\text{DPPV}}]$. These indicators were selected from a preliminary study, and were based on the information reported in Sect. 5.5. All the statistical indicators selected are nonrobust statistical metrics that help detect outliers. No central tendency indicators were used because YOB has a non-stationary time series. As this benchmark is univariate, the sensors' list comprises only one time series ($F = 1$).

In these experiments and in the next, 40% of each of the instances is for training and 60% for testing. The size of the N sliding window is 10 ($N = 10$). This value was chosen based on the internal characteristics of the dataset. The amount of windows K used for training varies between 57 and 67 ($K \in [57, 67]$) since the instances had different amounts of data. The cutoff factor used to remove potentially anomalous windows from the training was 10% ($\text{cut_factor} = 0.10$). Because the time series of the sub-benchmark A4 has concept change points, it was necessary to redo the learning stage constantly to relearn the new trends in the time series format. If this constant relearning is not put, the assertiveness rate is lower. Figure 6 illustrates the result obtained by the AOD method applied in one instance for each sub-benchmark A1 through A4. In this figure and the next ones, time series

are highlighted in blue, and the vertical green columns are labeled outliers in the benchmark. Rectangles at the bottom in orange are the columns where AOD detected outliers; red rectangles are outliers not detected by the method; yellow rectangles are the columns removed from the training; pink rectangles highlight the beginning of concept changes in the time series.

Table 5 presents the results obtained by the AOD method measured over several forms of evaluation. This table groups the results by sub-benchmark and presents the confusion matrix. Due to the large imbalance between outlier and non-outlier classes, the accuracy is not adequate, since the same weight is given for each hit. What is generally desired is a high Recall of the method and high Accuracy. When applied to the problem of anomaly detection in industrial machines, the Recall factor is more important than the Precision factor because it is preferable to announce an anomaly even if it does not occur (FP) than not detect it (FN). The F1-score and AUC are more suitable to verify the balanced performance of the method. Once the average performance is good, a high Recall is sought. The outliers of the A1 sub-benchmark are very difficult to detect, as shown by the F1-score of 48.6%. This difficulty can be attributed to real data with complex behavior, where there is no trend effect and seasonality. The sub-benchmark A4 also presents this difficulty, F1-score of 58.1%, but that comes from the concept changes in the series. In the A2 and A3 sub-benchmarks, the detection of outliers is relatively easy as revealed by the F1-score of over 90%. Finally, the AUC value obtained in sub-benchmarks A1 through A4 was over 80%.

The total number of False Positive (FP) in YOB was 540, while the number of False Negatives (FN) was 332 (see Table 5). These numbers show that there is a 62% higher chance of the algorithm accusing FP than FN. These values are an acceptable result considering that in predictive maintenance there is more permissiveness for FP rather than FN. FP means that an intervention will be planned before an actual breakdown while FN means the breakdown of the machine occurred.

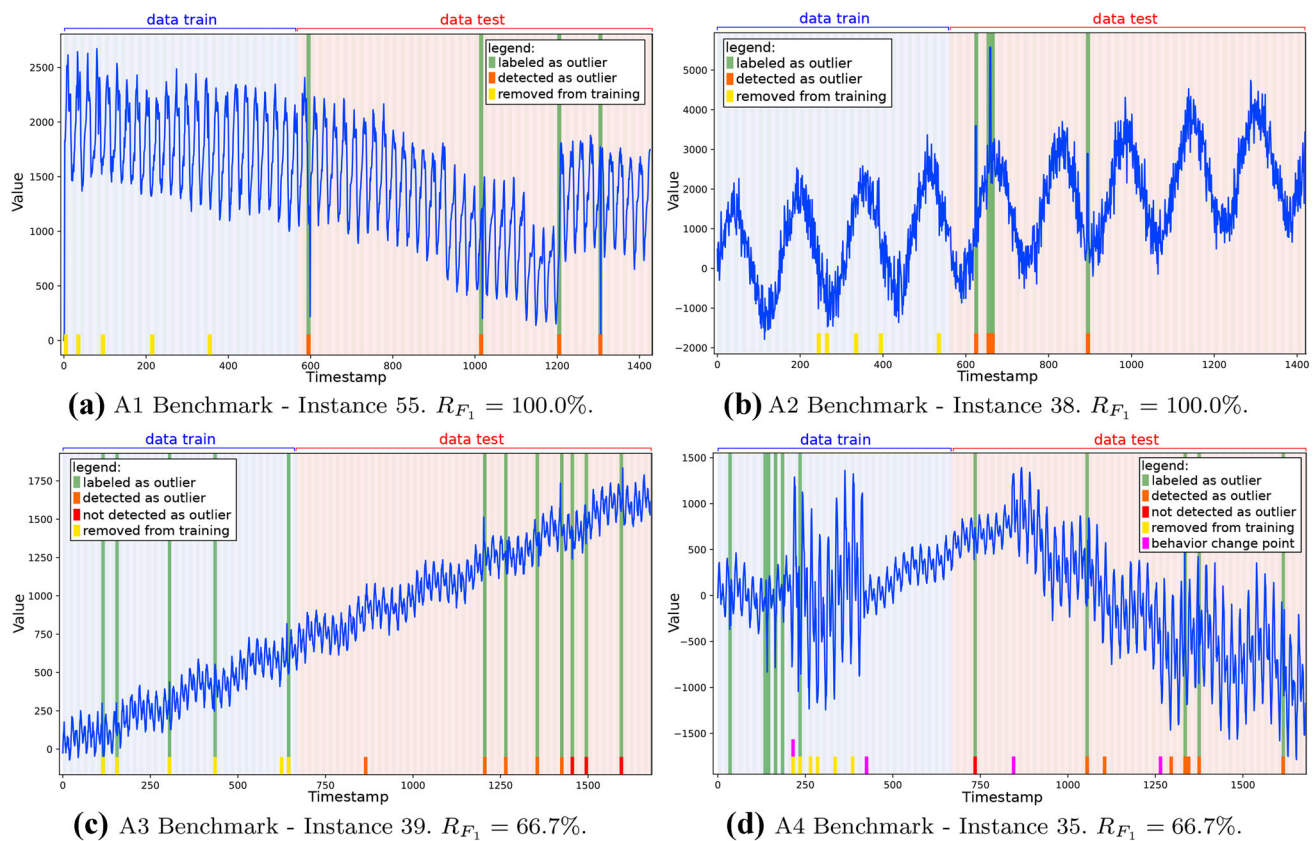


Fig. 6 Examples of time series for each sub-benchmark of YOB

Table 5 Evaluation of different metrics for the AOD method applied on YOB

Benchmark Sub-bench.	Confusion matrix [[TP, FP], [FN, TN]]	Different evaluation metrics				
		R_{rec} (%)	R_{pre} (%)	R_{acc} (%)	R_{F1} (%)	R_{auc} (%)
A1	[[160, 254], [84, 5201]]	65.6	38.7	94.1	48.6	80.2
A2	[[231, 0], [2, 8367]]	99.1	100.0	99.9	99.6	99.0
A3	[[493, 1], [76, 9530]]	86.6	99.8	99.2	92.8	94.9
A4	[[316, 285], [170, 9329]]	65.0	52.5	95.5	58.1	83.1
Overall	[[1210, 540], [332, 32427]]	78.5	69.1	97.5	77.5	89.3

The AOD method cannot be directly compared with other outlier detection methods like DeepAnT, Yahoo EGADS, and Twitter S-H-ESD because the methodologies are different. For example, DeepAnT, Yahoo EGADS, and Twitter S-H-ESD return for each point, whether it is an outlier or not. In contrast, the AOD method returns if the analyzed sliding data window contains an outlier value or not.

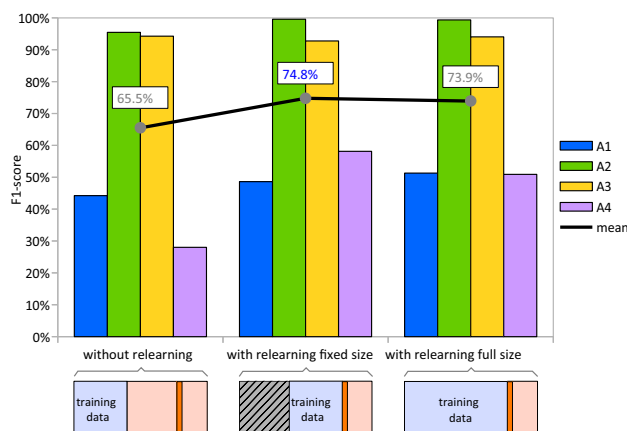
Any method must perform better than a naive estimation at least. Table 6 shows the results obtained by applied dummy methods on YOB. The evaluated dummy strategies are Positive, Negative, Uniform, and Stratified. The Positive strategy indicates that every point is an outlier, and the Negative indicates that all points are not outliers. There are equal chances for the Uniform strategy (50%) to mark the point as an outlier and as a regular point. Finally, the Stratified strategy

has a 5% chance to answer that it is an outlier and 95% chance to answer that it is not an outlier, where the value 5% was extracted from the fact that on average, the A1 to A4 benchmarks have 5% of windows with outliers values. The Uniform and Stratified methods were executed ten times because of their stochastic nature, and then the general average was reported. As shown in Table 6, the Positive method has better performance when analyzed over the recall and F1-score metrics. The Negative strategy has better performance when analyzed using accuracy and precision metrics. Finally, all dummy methods have equivalent performance when analyzed over AUC metrics.

Figure 7 shows the result obtained on YOB with different forms of the relearning procedure. In the first block of this figure, there are analyses in which the learning was done

Table 6 Evaluation of different metrics for dummy methods on YOB

Method Dummy	Benchmark Sub-bench.	Different evaluation metrics				
		R_{rec} (%)	R_{pre} (%)	R_{acc} (%)	R_{F_1} (%)	R_{auc} (%)
Positive	A1	100.0	4.5	4.5	8.6	50.0
Negative	A1	0.0	100.0	95.5	0.0	50.0
Uniform	A1	49.9	4.3	50.1	7.9	49.6
Stratified	A1	5.0	4.3	91.2	4.6	49.9
Positive	A2	100.0	2.7	2.7	5.3	50.0
Negative	A2	0.0	100.0	97.3	0.0	50.0
Uniform	A2	50.3	2.7	49.9	5.2	49.9
Stratified	A2	5.9	3.2	92.6	4.1	50.7
Positive	A3	100.0	5.6	5.6	10.7	50.0
Negative	A3	0.0	100.0	94.4	0.0	50.0
Uniform	A3	50.4	5.7	50.2	10.2	50.9
Stratified	A3	5.1	5.8	90.0	5.4	50.2
Positive	A4	100.0	4.8	4.8	9.2	50.0
Negative	A4	0.0	100.0	95.2	0.0	50.0
Uniform	A4	50.2	4.8	50.0	8.8	50.3
Stratified	A4	5.1	4.8	90.6	4.9	49.9

**Fig. 7** Applications of different forms of relearning on the AOD method in YOB

only in the first section of the time series, and there was no relearning procedure over time. In the second block, the relearning step was done to each new window of data, e.g., the training block goes sliding and adding new windows at the same time that it goes forgetting the same amount of windows at the beginning. In this approach, the percentage of training is always 40% of the observations. In the third block, we have analyzed where the training window grows as new data are processed. In this type of analysis, the method does not forget the initial data, and all new observations are added.

The results obtained without relearning were not good, mostly when there were points of change of concept. This behavior occurs because the model is trained with data in

a particular condition and does not understand when a new concept appears in the observations. Such a change causes a sequence of false positives or false negatives if there is no relearning of the model. By introducing the relearning fixed size, subset A4 obtained a 30.1% improvement in F1-score.

Finally, the average time required for the training of each of the instances by the AOD method was 27.3 ms. The average time needed to evaluate each of the test windows was 8.4 ms. Computational experiments were conducted on a computer with an Intel Core i5 8th generation processor with 8 GB of RAM.

5.2 Evaluation on NAB: univariate

The data set Numenta Anomaly Benchmark (NAB)² contains 58 univariate time series data with the anomaly points labeled in a separate file. It is subdivided into seven sub-benchmarks: artificialNoAnomaly (artNoAno), artificialWithAnomaly (artWiAno), realKnownCause (realKnCa), realAdExchange (realAdEx), realAWScloudwatch (realAWS), realTraffic and realTweets. The artificialNoAnomaly sub-benchmark contains five time series with artificial data and no anomaly. The artificialWithAnomaly sub-benchmark contains six time series of artificial data with some anomalies. The realKnownCause sub-benchmark contains seven time series of real data with anomalies points, where some of these anomalies are also outliers. The sub-benchmarks realAdExchange, realAWScloudwatch, realTraffic, and realTweets also contain several time series with anomalies points and/or

² <https://github.com/numenta/NAB>.

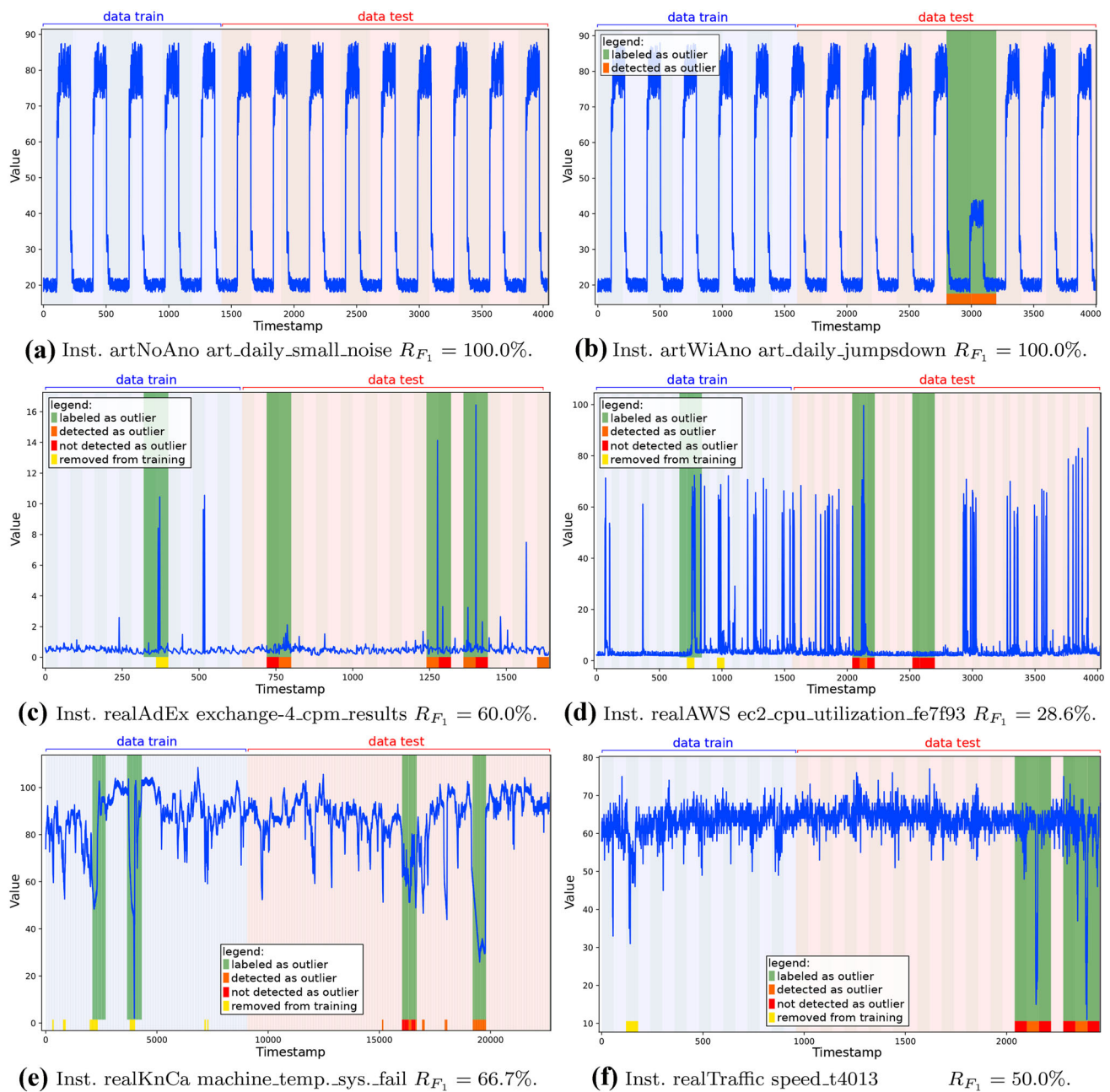


Fig. 8 Examples of time series for each NAB sub-benchmark

outliers. The realAWSCloudwatch sub-benchmark has, in addition to anomalies and outliers, concept change points in the time series.

For this benchmark, both anomalies and outliers are of interest. Thus, the features used were: M_{AMean} , M_{RMS} , M_{Median} , M_{IQM} , M_{STD} , M_{PPV} , M_{MAD} , M_{IQR} , M_{SV} , M_{KV} , M_{SVBQ} , M_{KVBO} , M_{DMAD} , M_{DIQR} , M_{DSVBQ} and M_{DKVBO} . Ten of the selected statistical indicators are robust statistical metrics (for anomaly detection), and the other six selected statistical indicators are nonrobust (for outliers detection). More on these selection in Sect. 5.5.

For the experiments, the data window size N varied between 40 and 240 ($N \in [40, 240]$). This N value was chosen based on the internal characteristics of each sub-benchmarks. The amount of K windows used for training varied from 6 to 151 ($K \in [6, 151]$) because the instances had different amounts of data. The cutoff factor used in the training was 10% ($\text{cut_factor} = 0.10$). Figure 8 illustrates the result obtained by the AOD method applied to an instance for each sub-benchmark.

Table 7 presents the results obtained by the AOD method on the NAB, grouped by sub-benchmark and measured over

several metrics. In the artificial sub-benchmark artificial-NoAnomaly, which has no anomaly or outlier and can be considered simple, F1-score of 100% was achieved. In the artificial sub-benchmark artificial-WithAnomaly, also simple, it was relatively easy to detect the anomalies, where an F1-score of 95.7% was achieved. Note that the recall in this sub-benchmark was 100%, e.g., all anomalies were detected correctly, but the accuracy was 91.7%, indicating false positives. In the sub-benchmark realKnownCause, the detection reached an F1-score of 50.3%. Of all the sub-benchmarks analyzed, what we obtained the worst performance was the realTweets, with an F1-score of 36.4%. The realAWSCloudwatch sub-benchmark also had a relatively low performance with an F1-score of 38.8% because it's a complex dataset with anomaly points, outliers, and concept change points. In all analyzed sub-benchmarks, the rate obtained from AUC was higher than 62%.

One of the factors that explain the low F1-score values obtained is how the NAB performs the anomaly labeling. Initially, a specialist identifies the anomalies' centers in each of the data instances and labels them as anomalous data. Then, 10% of the observations around this anomaly are also classified as anomalous due to the difficulty of establishing when an anomaly begins and when it ends [21]. Thus, many other observations are also classified as anomalies, even without being so.

The total number of FN in the NAB was 196, and the number of FP was 104 (see Table 7). These numbers show an 88% higher chance of the algorithm accusing an FN than an FP. These values can be explained by how anomalies are labeled in the NAB, where the algorithm does not have enough information to predict that it is an anomaly even if the benchmark has labeled the observation as one.

As with YOB, the AOD method cannot be directly compared with DeepAnT, Yahoo EGADS, and Twitter S-H-ESD methods in NAB due to differences in methodology. Thus, by working with data windows and not data points, the recall, precision, and F1-score metrics obtained a much better result than those reported by the literature approaches. In fact, the values of recall, accuracy, and F1-score published by the NAB literature techniques are low. Again, this can be explained by the way the anomaly label was made in the NAB.

Table 8 shows the results obtained by dummy methods when applied over NAB, following the same strategies used for YOB. The only difference is in the Stratified strategy, which now has a 10% chance to answer that it is an anomaly and 90% chance to answer that it is not an anomaly. The value 10% was taken because NAB sub-benchmarks have, on average, 10% of windows with anomalies. As observed in Table 8, the Positive method has better performance when analyzed the recall and F1-score metrics. The Negative method has better performance when analyzed the accuracy and preci-

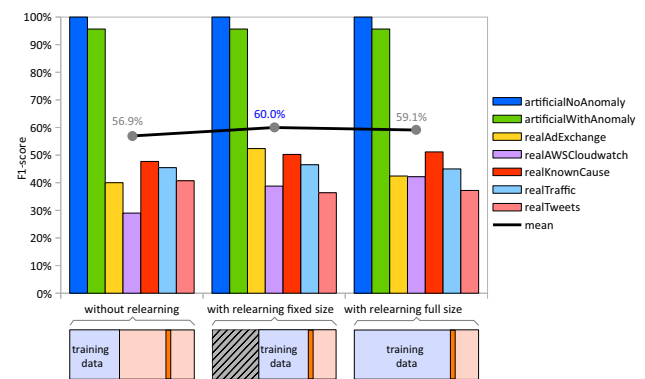


Fig. 9 Application analysis of different ways of relearning on the AOD method in the NAB

sion metrics. Finally, all dummy methods have equivalent performance when analyzed concerning AUC metrics. It is important to highlight that the AUC metric cannot be applied (N/A) when there is only one data class, as in the artificial-NoAnomaly sub-benchmark.

Figure 9 shows the result obtained on the NAB with the application of different ways of performing the relearned procedure. In the first block of this figure, there are analyses in which the learning was done only at the beginning of the time series, and there was no relearning procedure over time. In the second block, we have analyzed in which the relearned was done at each new data window. In the third block, we have analyzed in which the training window grows as new data are processed, and there is no forgetting of the initial data.

Results obtained without relearning were not satisfactory, mainly in those instances where there were points of concept change. By introducing the relearning fixed-size sub-benchmark, the realAWSCloudwatch got an improvement of 9.8% in F1-score. The average time needed for the training of each of the instances by the AOD method was 75.4 ms, with an average time of 26.6 ms per window.

5.3 Evaluation on synthetic dataset: multivariate

This subsection presents the computational results obtained on multivariate data generated synthetically from the AGOTS³ tool. Five artificial instances were generated with four time series. The first instance generated (multivariate-1) has no anomaly. The other four instances have anomalies such as shift, variance, and trend. This study validates the proposed method on synthetically generated multivariate data. The tool's flaws were labeled and were not used in training, only for evaluating the results.

Most of the flaws resemble anomalies, so the statistical indicators used were: M_{Median} , M_{IQM} , M_{MAD} , M_{IQR} , M_{DMAD} , M_{DIQR} , M_{DRMS} , M_{DVAR} . Thus, the number of met-

³ <https://github.com/KDD-OpenSource/agots>.

Table 7 Evaluation of different metrics for the AOD method in NAB subsets

Benchmark Sub-bench.	Confusion matrix [[TP, FP], [FN, TN]]	Different evaluation metrics				
		$R_{\text{rec}} (\%)$	$R_{\text{pre}} (\%)$	$R_{\text{acc}} (\%)$	$R_{F_1} (\%)$	$R_{\text{auc}} (\%)$
artNoAno	[[0, 0], [0, 55]]	100.0	100.0	100.0	100.0	N/A
artWiAno	[[11, 1], [0, 60]]	100.0	91.7	98.6	95.7	99.2
realAdEx	[[11, 9], [11, 113]]	50.0	55.0	86.1	52.4	66.9
realAWS	[[32, 33], [68, 553]]	32.0	49.2	85.3	38.8	62.9
realKnCa	[[50, 45], [54, 548]]	48.1	52.6	85.8	50.3	65.2
realTraffic	[[10, 4], [19, 124]]	34.5	71.4	85.4	46.5	64.5
realTweets	[[16, 12], [44, 408]]	26.7	57.1	88.3	36.4	63.2
Overall	[[130, 104], [196, 1861]]	39.9	55.6	86.9	46.4	70.3

Table 8 Evaluation of different metrics for dummy methods on NAB

Method Dummy	Benchmark Sub-bench.	Different evaluation metrics				
		$R_{\text{rec}} (\%)$	$R_{\text{pre}} (\%)$	$R_{\text{acc}} (\%)$	$R_{F_1} (\%)$	$R_{\text{auc}} (\%)$
Positive	artNoAno	100.0	0.0	0.0	0.0	N/A
Negative	artNoAno	100.0	100.0	100.0	100.0	N/A
Uniform	artNoAno	100.0	0.0	49.5	0.0	N/A
Stratified	artNoAno	100.0	0.0	89.8	0.0	N/A
Positive	artWiAno	100.0	15.3	15.3	26.5	50.0
Negative	artWiAno	0.0	100.0	84.7	0.0	50.0
Uniform	artWiAno	49.1	15.0	49.7	23.0	50.8
Stratified	artWiAno	9.1	16.5	78.8	11.3	50.2
Positive	realAdEx	100.0	15.3	15.3	26.5	50.0
Negative	realAdEx	0.0	100.0	84.7	0.0	50.0
Uniform	realAdEx	57.7	16.7	49.7	25.9	51.7
Stratified	realAdEx	10.9	17.2	78.2	13.2	50.8
Positive	realAWS	100.0	14.6	14.6	25.4	50.0
Negative	realAWS	0.0	100.0	85.4	0.0	50.0
Uniform	realAWS	49.7	14.5	50.0	22.4	49.9
Stratified	realAWS	8.9	13.9	78.4	10.8	49.5
Positive	realKnCa	100.0	14.9	14.9	26.0	50.0
Negative	realKnCa	0.0	100.0	85.1	0.0	50.0
Uniform	realKnCa	50.0	14.8	49.6	22.8	50.6
Stratified	realKnCa	8.9	13.3	77.8	10.7	49.4
Positive	realTraffic	100.0	18.5	18.5	31.2	50.0
Negative	realTraffic	0.0	100.0	81.5	0.0	50.0
Uniform	realTraffic	46.2	17.9	51.1	25.8	49.7
Stratified	realTraffic	9.3	18.1	75.7	12.2	50.3
Positive	realTweets	100.0	12.5	12.5	22.2	50.0
Negative	realTweets	0.0	100.0	87.5	0.0	50.0
Uniform	realTweets	51.5	12.7	49.7	20.4	50.6
Stratified	realTweets	8.7	10.3	79.4	9.4	49.1

rics selected is $J = 8$. It is important to note that six of the eight statistical indicators selected are robust statistical metrics that assist in anomaly detection. This dataset is multivariate and consists of four time series ($F = 4$). In the experiments' conduction, the data window size N was set to

60 ($N = 60$). The amount of K windows used for training was 16 ($K = 16$). The cutoff factor used in the training was 10 percent of the data windows (cut_factor = 0.10). Figure 10 shows the detections performed by the AOD method on an

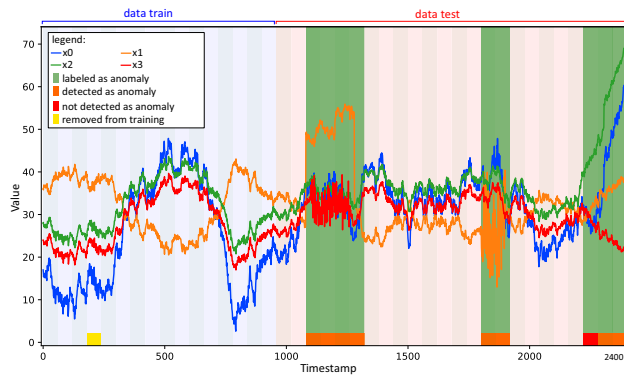


Fig. 10 Result of the AOD method applied on multivariate data (multivariate-5 instance) with four time series

instance of this multivariate benchmark, where an F1-score of 94.1% was reached.

Table 9 presents the evaluation of the AOD method over the five instances generated synthetically by the AGOTS tool. An average F1-score of 91.4% and an average AUC of 92.4% were obtained, which reveal a proper anomaly detection. Observing the confusion matrix, we can see that the false positive number was zero indicating that all the anomalies detected by the method are, in fact, anomalies.

Table 10 shows the results obtained by dummy strategies when applied to synthetic multivariate data. The Stratified method, independent of any value read in the input, has a 16% chance to answer that it is an anomaly and an 84% chance to answer that it is not an anomaly. The value 16% was taken because this dataset has 16% of windows with anomalous values on average.

The average time required for the training of each of the instances by the AOD method was 67.6 ms. The average time needed to evaluate each of the test windows of the instances was 5.55 ms.

5.4 Evaluation on case study: multivariate

This subsection presents the computational results obtained from a case study conducted in a vegetal oil production indus-

Table 10 Evaluation of different metrics for dummy methods in the synthetic multivariate dataset

Method	Different evaluation metrics				
	R_{rec} (%)	R_{pre} (%)	R_{acc} (%)	R_{F1} (%)	R_{auc} (%)
Dummy					
Positive	100.0	15.8	15.8	27.3	50.0
Negative	0.0	100.0	84.2	0.0	50.0
Uniform	47.9	15.5	50.0	23.4	48.3
Stratified	13.2	12.0	71.6	12.5	48.5

try. This machine is used to break grains and consists of two motors that rotate two pairs of cylinders, one upper and one lower. This study considers two time series that represent electrical current data of two motors. These sensors can be correlated depending on the nature of the machine. The grains that enter the machine from the top demand more energy to be broken, so the value of one of the sensors has to be higher. Thus, this case study validates the proposed method on a multivariate time series. Here occurs the detection of a fault called lack of load, which is considered an outlier in terms of time series. An expert labeled the failures, but the labels were not used during training, only during results evaluation.

Since the failures resemble outliers, the statistical indicators used were: M_{STD} , M_{PPV} , M_{DSTD} , M_{DPPV} . Thus, the number of metrics selected is $J = 4$. All the statistical indicators selected are nonrobust statistical metrics that assist in detecting outliers. As this benchmark is bivariate, the sensor list comprises two time series ($F = 2$).

In the following experiments, the data window size N was set to 60 ($N = 60$). The amount of K windows used for training was 6312 ($K = 6312$) with a cutoff factor of 10% of the data windows (cut_factor = 0.10). Figure 11 shows the detections performed by the AOD method in that case study, where an F1-score of 90.3% was reached.

Table 11 presents the evaluation of the AOD method over the case study, yielding an F1-score of 90.3% and an AUC of 91.2%. Observing the confusion matrix, one can see that the false positive number was zero, and the false negative number was three.

Table 9 Evaluation of different metrics for the AOD method applied on a synthetic multivariate dataset

Benchmark Instance	Confusion Matrix [[TP, FP], [FN, TN]]	Different evaluation metrics				
		R_{rec} (%)	R_{pre} (%)	R_{acc} (%)	R_{F1} (%)	R_{auc} (%)
Multivariate-1	[[0, 0], [0, 24]]	100.0	100.0	100.0	100.0	N/A
Multivariate-2	[[2, 0], [0, 22]]	100.0	100.0	100.0	100.0	100.0
Multivariate-3	[[3, 0], [1, 20]]	75.0	100.0	95.8	85.7	87.5
Multivariate-4	[[3, 0], [1, 20]]	75.0	100.0	95.8	85.7	87.5
Multivariate-5	[[8, 0], [1, 15]]	88.8	100.0	95.8	94.1	94.4
Overall	[[16, 0], [3, 101]]	84.2	100.0	97.5	91.4	92.4

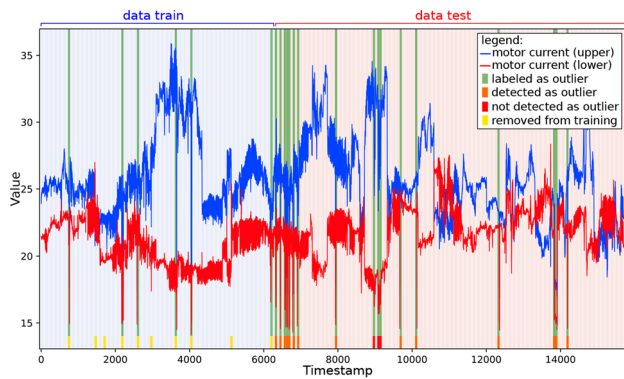


Fig. 11 Result of the AOD method when it was applied in the case study with two time series

Table 12 shows the results obtained by the dummy strategies when applied over the case study. The Stratified method, regardless of any value read in the input, has a 2% chance of responding that it is an anomaly and a 98% chance of responding that it is not an anomaly. The value 2% was extracted from the fact that, on average, this case study has 2% of windows with anomalous values.

The average time required for the training of each of the instances was 76.0 ms, and the average time needed to evaluate each of the instances' test windows was 10.1 ms.

5.5 Study on the behavior of statistical features

This study was performed in order to measure the impact of each of the statistical features presented in Table 3 in terms of F1-score. Figure 12 shows the performance on the YOB, NAB, synthetic dataset, and case study benchmarks. On the x-axis of Fig. 12a, there are the 68 implemented metrics. The F1-score for the YOB benchmark is on the y-axis. Figure 12b shows the NAB benchmark evaluation, and in Fig. 12c there is the evaluation for the synthetic dataset. Finally, in Fig. 12d we have the evaluation for the case study.

Table 13 presents an overview of available statistical estimators organized by benchmarks. Statistical features are grouped by category, and the table values are in terms of F1-score.

For YOB, the statistical features that got the best F1-score were the nonrobust ones. This observation can be justified because YOB contains mainly outliers, and the nonrobust estimators are sensitive to outlier values. Still, concerning YOB, the metric categories that obtained the best per-

Table 12 Evaluation of different metrics for dummy methods in the case study

Method	Different evaluation metrics				
	R_{rec} (%)	R_{pre} (%)	R_{acc} (%)	R_{F1} (%)	R_{auc} (%)
Positive	100.0	10.8	10.8	19.4	50.0
Negative	0.0	100.0	89.2	0.0	50.0
Uniform	52.9	11.3	50.2	18.6	51.4
Stratified	3.5	16.8	88.0	5.7	50.9

mance were those of dispersion since an outlier brings with it a large dispersion of data. Regarding the signal origin, the performance is improved when the statistical features are those in the derivative domain.

Regarding the results obtained on NAB, the robust and nonrobust statistical estimators obtained an equivalent performance. This result is justified because NAB contains both anomalies and outliers. Thus, the anomalies are mainly discovered by the robust estimators, and the nonrobust ones reveal the outliers. Still, the statistical feature category that obtained the NAB's best performance is the central trend measures. This happens because when an anomaly occurs, a significant shift in the centrality of the data occurs. Regarding the signal origin, the performance is improved when the original signal features are used, that is, without applying the derivative.

Regarding the results obtained from the case study, the nonrobust statistical estimators obtained superior performance. This result is justified because the failures present many similarities with the detection of outliers in the case study. The statistical feature categories that obtained the best performance were those of dispersion. About the signal origin, the performance is improved when metrics are used in the derivative domain.

The results of Sects. 5.1, 5.2, 5.3 and 5.4 used more than one type of statistical feature in combination. The use of combined features, in general, helps in the detection of faults with varied behaviors. No study has been conducted to find the best features that should be used for each particular benchmark. Thus, studying the nature of the data and signal helps select which features to use in experiments.

A set of images containing the results obtained for all instances evaluated in the YOB, NAB, synthetic dataset,

Table 11 Evaluation of different metrics for the AOD method applied on case study

Benchmark	Confusion matrix [[TP, FP], [FN, TN]]	Different evaluation metrics			
		R_{rec} (%)	R_{pre} (%)	R_{acc} (%)	R_{F1}
Case study	[[14, 0], [3, 141]]	82.4	100.0	98.1	90.3

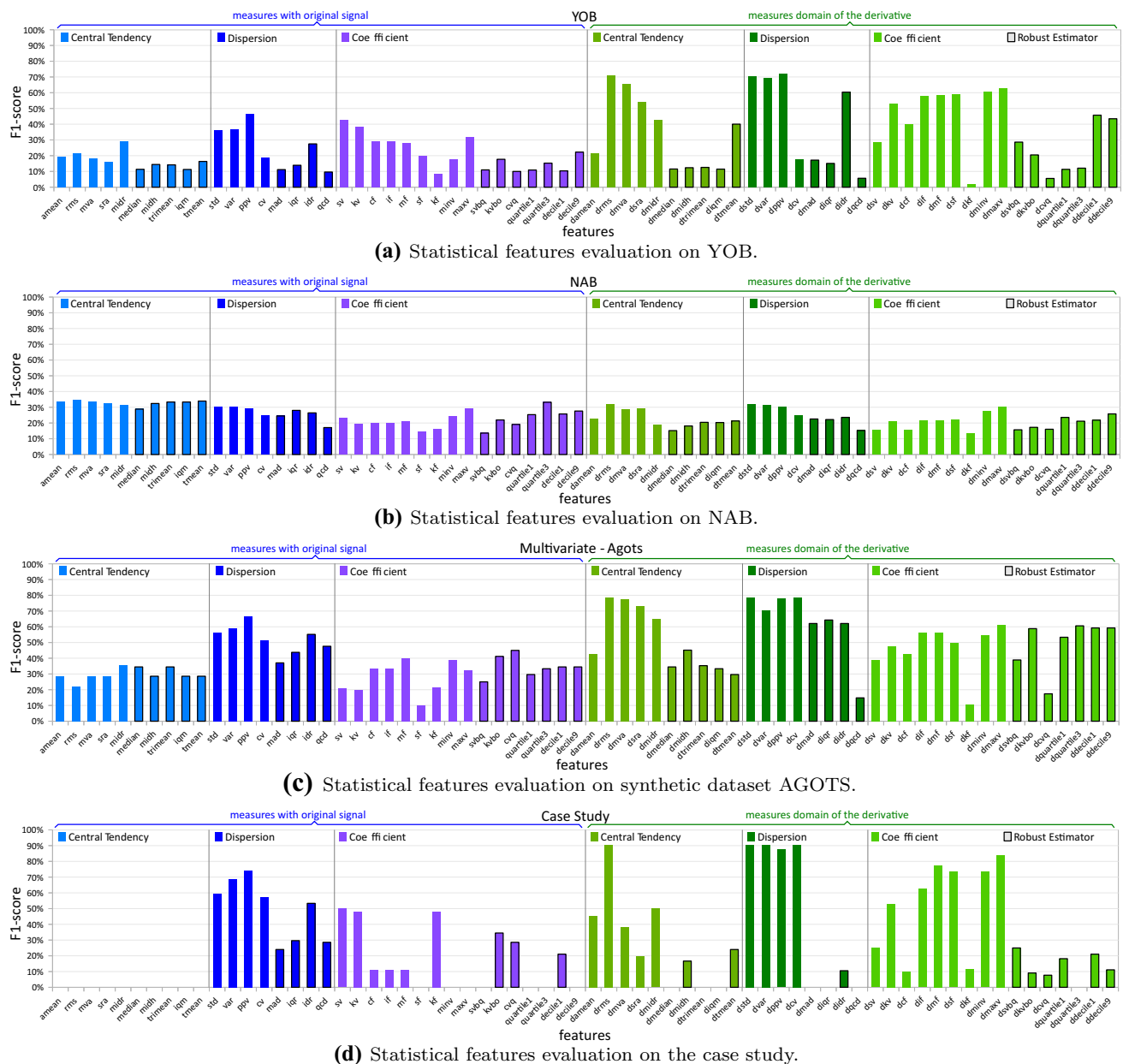


Fig. 12 Evaluation of the impact of statistical features in relation to the F1-score

and case study benchmarks are found at GitHub at. ⁴ These images show all the analyzed time series and the locations where the anomalies and outliers were detected. This repository also contains artificially generated instance files.

6 Discussion and conclusions

This paper presented a new method, called Anomaly and Outlier Detector (AOD) for detecting anomalies and out-

liers in time series. The way of learning is unsupervised, and the speed of training and evaluation is quite fast due to its low complexity. The method is designed to run in real time (online) but can also run offline. It supports univariate or multivariate data evaluation and uses a data sliding window from which statistical features are extracted and analyzed. Due to the way the method was developed, it can easily detect anomalies or outliers just by changing the statistical indicators used. This functionality is critical, since it extends the detection capability of the proposed method to two slightly different domains. It has been shown that the proposed method can handle concept change points in time

⁴ <https://github.com/jesimar/aod>.

Table 13 Evaluation of statistical features by category

Measures by features category	F1-score (%)		Dataset AGOTS	Case study	Average		
	YOB	NAB					
Central tendency (nonrobust)	20.9	33.0	28.7	0.0	20.6		
Central tendency (robust)	13.5	32.4	30.9	0.0	19.2	19.9	
Dispersion (nonrobust)	34.5	28.4	58.3	64.8	46.5		
Dispersion (robust)	15.5	24.0	45.9	33.9	29.8	38.2	
Coefficient (nonrobust)	27.2	20.8	27.8	20.0	23.9		
Coefficient (robust)	13.9	23.8	34.7	12.0	21.1	22.5	26.9
Derivative central tendency (nonrobust)	51.0	26.0	67.4	48.7	48.3		
Derivative central tendency (robust)	17.6	19.1	35.6	8.1	20.1	34.2	
Derivative dispersion (nonrobust)	57.2	29.4	76.4	89.6	63.2		
Derivative dispersion (robust)	24.6	20.9	50.8	2.6	24.7	43.9	
Derivative coefficient (nonrobust)	47.0	20.9	46.5	52.2	41.7		
Derivative coefficient (robust)	23.9	20.2	49.7	13.2	26.7	34.2	37.4

series data by simply relearning the history of data as new data are collected. This relearning is necessary because the former learned model becomes inconsistent with the new data by incorporating changes in the time series. The proposed method helps in the realization of predictive maintenance with focus on the diagnostic part.

The method was evaluated in two univariate benchmarks: for outliers detection (YOB) and for anomalies detection (NAB). The method was also evaluated with synthetically generated multivariate data and a real case study with data collected from a vegetal oil production industry. The experiments revealed that the sliding window size should be small for outlier detection. It was also found that a relearn procedure is required to handle concept changes in data as new observations are acquired.

AOD was compared with four dummy approaches, and in all of them, it obtained a superior result in terms of F1-score and AUC. In terms of accuracy, AOD only loses at A1 at YOB and realAWSCloudwatch at NAB, in which the dummy Negative strategy obtained superior results. However, the dummy strategy brought an F1-score equal to 0% in these cases. The average time for training was, in general, less than 76 ms and the average time in the evaluation was less than 27 ms. Using statistical features with the signal in the derivative domain showed, in general, a better strategy than with the original signal. Such a choice results in an overall improvement in the F1-score of more than 10%. Results in terms of AUC for YOB were 89.3%, 70.3% for NAB, for the synthetic dataset generated by AGOTS was 92.4%, and 91.2% for the case study. These high AUC values reveal the potential of the proposed method.

Because it is a flexible method, AOD can be replicated in different industrial applications related to predictive maintenance in detecting faults in industrial machines. This method

is deployed to monitor a set of machines in a large soybean oil production industry.

Acknowledgements The authors of this work thank the SENAI Institute of Innovation in Embedded Systems for its support in research development.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Ahmad, S., Lavin, A., Purdy, S., Agha, Z.: Unsupervised real-time anomaly detection for streaming data. *Neurocomputing* **262**, 134–147 (2017)
2. Amarbayasgalan, T., Pham, V.H., Theera-Umpon, N., Ryu, K.H., et al.: Unsupervised anomaly detection approach for time-series in multi-domains using deep reconstruction error. *Symmetry* **12**(8), 1251 (2020)
3. Anderson, M.J., Thompson, A.A.: Multivariate control charts for ecological and environmental monitoring. *Ecol. Appl.* **14**(6), 1921–1935 (2004)
4. Attoui, I., Oudjani, B., Boutasseta, N., Fergani, N., Bouakkaz, M.S., Bouraiou, A.: Novel predictive features using a wrapper model for rolling bearing fault diagnosis based on vibration signal analysis. *Int. J. Adv. Manuf. Technol.* **106**(7–8), 3409–3435 (2020)
5. Blanchard, B.S., Verma, D.C., Peterson, E.L.: *Maintainability: A Key to Effective Serviceability and Maintenance Management*, vol. 13. Wiley, Hoboken (1995)
6. Blázquez-García, A., Conde, A., Mori, U., Lozano, J.A.: A review on outlier/anomaly detection in time series data. *arXiv preprint arXiv:2002.04236* (2020)
7. Carvalho, T.P., Soares, F.A., Vita, R., Francisco, R.D.P., Basto, J.P., Alcalá, S.G.: A systematic literature review of machine learning methods applied to predictive maintenance. *Comput. Ind. Eng.* **137**, 106024 (2019)

8. Xy, Chen, Yy, Zhan: Multi-scale anomaly detection algorithm based on infrequent pattern of time series. *J. Comput. Appl. Math.* **214**(1), 227–237 (2008)
9. Dias, C.G., da Silva, L.C., Chabu, I.E.: Fuzzy-based statistical feature extraction for detecting broken rotor bars in line-fed and inverter-fed induction motors. *Energies* **12**(12), 2381 (2019)
10. Ding, N., Ma, H., Gao, H., Ma, Y., Tan, G.: Real-time anomaly detection based on long short-term memory and Gaussian mixture model. *Comput. Electr. Eng.* **79**, 106458 (2019)
11. Fernandez-Temprano, M., Gardel-Sotomayor, P., Duque-Perez, O., Morinigo-Sotelo, D.: Broken bar condition monitoring of an induction motor under different supplies using a linear discriminant analysis. In: 9th IEEE International Symposium on Diagnostics for Electric Machines, Power Electronics and Drives (SDEMPED), pp. 162–168. IEEE (2013)
12. Hochenbaum, J., Vallis, O.S., Kejariwal, A.: Automatic anomaly detection in the cloud via statistical learning. *arXiv preprint arXiv:1704.07706* (2017)
13. ISO: Condition monitoring and diagnostics of machines-general guidelines. Standard, International Organization for Standardization (2018)
14. Jimenez, J.J.M., Schwartz, S., Vingerhoeds, R., Grabot, B., Salaün, M.: Towards multi-model approaches to predictive maintenance: a systematic literature survey on diagnostics and prognostics. *J. Manuf. Syst.* **56**, 539–557 (2020)
15. Kagermann, H., Wahlster, W., Helbig, J.: Recommendations for implementing the strategic initiative industrie 4.0. Report, ACATECH-National Academy of Science and Engineering. <http://alvarestech.com/temp/tcn/CyberPhysicalSystems-Industrial4-0.pdf> (2013)
16. Kim, T.H., White, H.: On more robust estimation of skewness and kurtosis. *Financ. Res. Lett.* **1**(1), 56–73 (2004)
17. Laptev, N., Amizadeh, A., Billawala, Y.: Yahoo labs news: announcing a benchmark dataset for time series anomaly detection [online blog]. <https://research.yahoo.com/news/announcing-benchmark-dataset-time-series-anomaly-detection> (2015a)
18. Laptev, N., Amizadeh, S., Flint, I.: Generic and scalable framework for automated time-series anomaly detection. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1939–1947 (2015b)
19. Lavin, A., Ahmad, S.: Evaluating real-time anomaly detection algorithms—the numenta anomaly benchmark. In: IEEE 14th International Conference on Machine Learning and Applications (ICMLA), pp. 38–44. IEEE (2015)
20. Lee, J., Bagheri, B., Kao, H.: A cyber-physical systems architecture for industry 4.0-based manufacturing systems. *Manuf. Lett.* **3**, 18–23 (2015)
21. Munir, M., Siddiqui, S.A., Dengel, A., Ahmed, S.: DeepAnT: a deep learning approach for unsupervised anomaly detection in time series. *IEEE Access* **7**, 1991–2005 (2018)
22. Nemeth, T., Ansari, F., Sihn, W.: A maturity assessment procedure model for realizing knowledge-based maintenance strategies in smart manufacturing enterprises. *Procedia Manuf.* **39**, 645–654 (2019)
23. Olive, D.J.: Applied robust statistics. Preprint M-02-006 (2008)
24. Shukla, S., Yadav, R., Sharma, J., Khare, S.: Analysis of statistical features for fault detection in ball bearing. In: IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), pp. 1–8. IEEE (2015)
25. Stern, S., Farioli, S., Eisenschmidt, E., Morachioli, S., Kienzler, C., Jeske, D., Schaal, N.: The future of maintenance for distributed fixed assets. Technical report, McKinsey Company In collaboration with Lumics (2020)
26. Sullivan, G. P, Pugh, R., Melendez, A. P.: Operations and maintenance best practices—a guide to achieving operational efficiency. Technical report, Pacific Northwest National Lab., Richland, WA (US) (2002)
27. Susto, G.A., Schirru, A., Pampuri, S., McLoone, S., Beghi, A.: Machine learning for predictive maintenance: a multiple classifier approach. *IEEE Trans. Ind. Inf.* **11**(3), 812–820 (2014)
28. Wang, J., Zhao, R., Gao, R.X.: Probabilistic transfer factor analysis for machinery autonomous diagnosis cross various operating conditions. *IEEE Trans. Instrum. Meas.* **69**, 5335–5344 (2020)
29. Wu, J., Zeng, W., Yan, F.: Hierarchical temporal memory method for time-series-based anomaly detection. *Neurocomputing* **273**, 535–546 (2018)
30. Xia, Z., Xia, S., Wan, L., Cai, S.: Spectral regression based fault feature extraction for bearing accelerometer sensor signals. *Sensors* **12**(10), 13694–13719 (2012)
31. Yaacob, A. H., Tan, I. K., Chien SF, Tan, H. K.: Arima based network anomaly detection. In: Second International Conference on Communication Software and Networks, pp. 205–209. IEEE (2010)
32. Zheng, D., Li, F., Zhao, T.: Self-adaptive statistical process control for anomaly detection in time series. *Expert Syst. Appl.* **57**, 324–336 (2016)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.